

RTL Design Sherpa

RTL AMBA APB4 Module Reference — Rev 0.91

October 2026

Table of Contents

1 APB (Advanced Peripheral Bus) Modules....14
1.1 Overview....14
1.1.1 Protocol Scope: APB4, not APB5....14
1.1.2 Core Protocol Components....14
1.1.3 Testbench Utilities....15
1.1.4 Interconnect Components....15
1.1.5 Clock-Gated Variants....15
1.1.6 Key Features....16
1.2 Functional Description....16
1.2.1 APB Transfer Phases....16
1.2.2 Signal Descriptions....17
1.3 Usage Example....17
1.3.1 Using APB Master....17
1.3.2 Using APB Slave....18
1.4 Design Notes....19
1.4.1 Command/Response Architecture....19
1.4.2 Monitor Bus Protocol....19
1.5 Related Modules....20
1.6 Testing....20
1.6.1 WaveDrom Tests....20
1.6.2 Test Coverage Gaps....21
1.7 References....21

1.7.1 Specifications....21
1.7.2 Papers and textbooks....21
1.7.3 Source Code....22
1.7.4 Test Documentation....22
1.8 Navigation....23
2 apb4_master....24
2.1 Overview....24
2.2 Parameters....24
2.3 Ports....24
2.3.1 Clock and Reset....25
2.3.2 APB Master Interface....26
2.3.3 Command Interface....26
2.3.4 Response Interface....27
2.4 Functional Description....28
2.4.1 Block diagram....28
2.4.2 APB Protocol Implementation....28
2.4.3 Command Processing Flow....28
2.4.4 Key Features....28
2.4.5 Protection Attributes (PPROT)....29
2.4.6 Write Strobe Support....29
2.4.7 Error Handling....29
2.5 Timing Characteristics....30
2.5.1 APB Transaction Timing....30
2.5.2 Performance Metrics....30

2.6 Waveforms....31
2.6.1 Scenario 1: Basic Write Transaction....31
2.6.2 Scenario 2: Basic Read Transaction....32
2.6.3 Scenario 3: Back-to-Back Writes....32
2.6.4 Scenario 4: Back-to-Back Reads....32
2.6.5 Scenario 5: Write-to-Read Transition....33
2.6.6 Scenario 6: Read-to-Write Transition....33
2.7 Usage Examples....33
2.8 Design Notes....34
2.8.1 FIFO Depth Selection....34
2.8.2 Clock Gating....35
2.8.3 Synthesis Considerations....35
2.9 Related Modules....35
2.10 Testing....36
2.10.1 Protocol Compliance....36
2.10.2 Interface Verification....36
2.10.3 Performance Verification....36
2.11 References....36
2.12 Navigation....37
3 apb4_master_cg....38
3.1 Overview....38
3.2 Parameters....38
3.2.1 Derived Parameters (do not override)....39
3.3 Ports....39

3.4 Functional Description....40
3.4.1 Block diagram....40
3.4.2 Wake-Up Condition....40
3.4.3 Wake-Up Latency....40
3.5 Timing Characteristics....41
3.6 Usage Examples....41
3.7 Design Notes....42
3.8 Related Modules....42
3.9 Testing....42
3.10 References....42
3.11 Navigation....43
4 apb4_master_stub....44
4.1 Overview....44
4.2 Parameters....44
4.3 Ports....44
4.3.1 Clock and Reset....45
4.3.2 APB Master Interface....46
4.3.3 Packed Command Interface....46
4.3.4 Packed Response Interface....46
4.4 Functional Description....47
4.4.1 Block diagram....47
4.4.2 Packet Interface....47
4.4.3 First/Last Framing....48
4.4.4 Packet Format Is Not Symmetric With apb4_slave_stub....48

4.4.5 Operation....48
4.5 Timing Characteristics....48
4.6 Usage Examples....49
4.7 Design Notes....50
4.7.1 Testbench Usage....50
4.7.2 Packed Interface Benefits....50
4.8 Related Modules....50
4.9 Testing....50
4.10 References....50
4.11 Navigation....51
5 apb4_monitor....52
5.1 Overview....52
5.2 Parameters....52
5.3 Ports....53
5.3.1 Clock and Reset....55
5.3.2 Command Interface Monitoring....55
5.3.3 Response Interface Monitoring....56
5.3.4 Configuration - Error Detection....56
5.3.5 Configuration - Performance Monitoring....56
5.3.6 Configuration - Debug....57
5.3.7 Configuration - Thresholds....57
5.3.8 Monitor Bus Interface....58
5.3.9 Status Outputs....58
5.4 Functional Description....59

5.4.1 Block diagram....59
5.4.2 Transaction Monitoring....60
5.4.3 Event Detection....60
5.4.4 Monitor Packet Format....60
5.5 Timing Characteristics....61
5.6 Usage Examples....61
5.7 Design Notes....62
5.7.1 Transaction Tracking....62
5.7.2 Packet Generation....62
5.7.3 Performance Considerations....63
5.8 Related Modules....63
5.9 Testing....63
5.10 References....63
5.11 Navigation....64
6 apb4_slave....65
6.1 Overview....65
6.2 Parameters....65
6.3 Ports....65
6.3.1 Clock and Reset....66
6.3.2 APB Slave Interface....67
6.3.3 Command Interface....67
6.3.4 Response Interface....68
6.4 Functional Description....68
6.4.1 Block diagram....68

6.4.2 APB Protocol Implementation..... 69
6.4.3 State Transitions..... 69
6.4.4 Orphan-Response Guard..... 70
6.4.5 Command/Response Flow..... 70
6.4.6 Buffering Architecture..... 70
6.4.7 Key Features..... 70
6.4.8 APB Transaction Timing..... 70
6.5 Timing Characteristics..... 71
6.5.1 Transaction Latency..... 71
6.5.2 Performance Metrics..... 71
6.6 Waveforms..... 72
6.6.1 Scenario 1: Basic Write Transaction..... 72
6.6.2 Scenario 2: Basic Read Transaction..... 72
6.6.3 Scenario 3: Back-to-Back Writes..... 73
6.6.4 Scenario 4: Back-to-Back Reads..... 73
6.6.5 Scenario 5: Write-to-Read Transition..... 73
6.6.6 Scenario 6: Read-to-Write Transition..... 73
6.6.7 Scenario 7: Error Response..... 74
6.7 Usage Examples..... 74
6.8 Design Notes..... 75
6.8.1 Buffer Depth Selection..... 75
6.8.2 Clock Domain Optimization..... 75
6.8.3 Synthesis Considerations..... 76
6.9 Related Modules..... 76

6.10 Testing....76
6.10.1 Protocol Compliance....76
6.10.2 Buffer Verification....77
6.10.3 Backend Integration....77
6.11 References....77
6.12 Navigation....78
7 apb4_slave_cdc....79
7.1 Overview....79
7.1.1 Key Features....79
7.2 Parameters....79
7.3 Ports....80
7.4 Functional Description....82
7.4.1 Block diagram....82
7.4.2 Clock Domains....82
7.4.3 CDC Structure....83
7.4.4 Reset Behavior....83
7.4.5 Why Not the Previous 2-Phase Handshake....84
7.5 Design Notes....84
7.6 Timing Characteristics....85
7.6.1 Maximum Clock Ratio....85
7.6.2 Timing Constraints....85
7.7 Waveforms....85
7.7.1 Scenario 1: Write Transaction with CDC....85
7.7.2 Scenario 2: Read Transaction with CDC....86

7.7.3 Scenario 3: Back-to-Back Writes with CDC....86
7.7.4 Scenario 4: Back-to-Back Reads with CDC....87
7.7.5 Scenario 5: Write-to-Read Transition with CDC....87
7.7.6 Scenario 6: Read-to-Write Transition with CDC....87
7.7.7 Scenario 7: Error Response with CDC....88
7.8 Usage Examples....88
7.9 Related Modules....89
7.10 Testing....89
7.11 References....90
7.12 Navigation....90
8 apb4_slave_cdc_cg....91
8.1 Overview....91
8.2 Parameters....92
8.2.1 Derived Parameters (do not override)....92
8.3 Ports....92
8.3.1 Clock-gating status outputs....93
8.4 Functional Description....95
8.4.1 Block diagram....95
8.5 Timing Characteristics....96
8.6 Usage Examples....96
8.7 Design Notes....97
8.8 Related Modules....97
8.9 Testing....97
8.10 References....98

8.11 Navigation....99
9 apb4_slave_cg....100
9.1 Overview....100
9.1.1 Key Differences from Base Module....100
9.1.2 When to Use the Clock-Gated Variant....100
9.2 Parameters....100
9.2.1 Derived Parameters (do not override)....101
9.3 Ports....101
9.4 Functional Description....102
9.4.1 Block diagram....102
9.4.2 Single Gating Domain....102
9.4.3 Wake-Up Condition....102
9.4.4 Gating State Machine....103
9.4.5 Wake-Up Latency....103
9.5 Timing Characteristics....103
9.6 Usage Examples....104
9.6.1 Aggressive Gating (Bursty Traffic)....104
9.6.2 Conservative Gating....104
9.6.3 Clock Gating Disabled (Functional Verification)....105
9.7 Design Notes....105
9.7.1 Power Savings....105
9.7.2 Gating Status Signal....105
9.7.3 Synthesis Considerations....105
9.8 Related Modules....106

9.9 Testing....106
9.9.1 Clock Gating in Simulation....106
9.9.2 Power Analysis Verification....106
9.10 References....107
9.11 Navigation....107
10 apb4_slave_stub....108
10.1 Overview....108
10.2 Parameters....108
10.3 Ports....108
10.3.1 Clock and Reset....109
10.3.2 APB Slave Interface....109
10.3.3 Packed Command Interface....110
10.3.4 Packed Response Interface....110
10.4 Functional Description....111
10.4.1 Block diagram....111
10.4.2 Packet Interface....111
10.4.3 Packet Format Is Not Symmetric With apb4_master_stub....111
10.4.4 Difference From apb4_slave....112
10.4.5 Operation....112
10.5 Timing Characteristics....112
10.6 Usage Examples....113
10.7 Design Notes....114
10.7.1 Testbench Usage....114
10.7.2 Response Generation....114

10.7.3 Packed Interface Benefits....114
10.8 Related Modules....114
10.9 Testing....114
10.10 References....114
10.11 Navigation....115

1 APB (Advanced Peripheral Bus) Modules

Location: `rtl/amba/apb4/` **Test Location:** `val/amba/` **Status:** Production Ready

1.1 Overview

The APB subsystem provides a complete implementation of the ARM AMBA 4 APB (Advanced Peripheral Bus) protocol: masters, slaves, monitors, interconnect components, and testbench utilities. APB is the simple, low-power peripheral bus you hang low-bandwidth peripherals off a system bus with — a two-cycle handshake and a minimal set of control signals. That simplicity is the whole point.

1.1.1 Protocol Scope: APB4, not APB5

Every module in `rtl/amba/apb4/` implements **AMBA 4 APB (APB4)**: `PSEL`, `PENABLE`, `PREADY`, `PADDR`, `PWRITE`, `PWDATA`, `PSTRB`, `PPROT`, `PRDATA`, `PSLVERR`.

The APB5 additions — `PWAKEUP`, `PAUSER`/`PWUSER`/`PRUSER`/`PBUSER`, and the optional parity signals — are **not** present on these modules. They live in a separate module family:

Family	RTL	Documentation
APB4 (this book)	`rtl/amba/apb4/`	`docs/markdown/rtl-amba/apb4/`
APB5	`rtl/amba/apb5/`	APB5 Modules

Use `apb5_slave` / `apb5_master` (and their `_cg` / `_cdc` variants) when APB5 signalling is required. The two families are otherwise architecturally identical.

The modules in this book fall into four groups.

1.1.2 Core Protocol Components

Module	Description	Documentation	Status
apb4_master	Full-featured APB master with command/response interface	apb4_master.md	Documented
apb4_slave	Complete APB slave with buffered cmd/rsp interface	apb4_slave.md	Documented
apb4_slave_	APB slave with clock	apb4_slave_cdc.md	Documented

Module	Description	Documentation	Status
cdc	domain crossing support		
apb4_monitor	Transaction monitoring with 128-bit monitor bus + 64-bit timestamp	apb4_monitor.md	Documented

Note: `apb4_monitor.sv` lives HERE in `rtl/amba/apb4/` — the protocol monitors stay with the protocol they wrap; only the monitor CORE pieces live in `rtl/amba/ocelus/`. Its specification is apb4_monitor.md in this book.

1.1.3 Testbench Utilities

Module	Description	Documentation	Status
apb4_master_stub	Lightweight APB master for testbench integration	apb4_master_stub.md	Documented
apb4_slave_stub	Lightweight APB slave for testbench integration	apb4_slave_stub.md	Documented

1.1.4 Interconnect Components

Module	Description	Documentation	Status
apbx_xbar_1to1 / **2to1** / **1to4** / **2to4** / **2to2_mixed**	Generated fixed-configuration crossbars	../apbx/README.md -> component MAS/HAS	Documented

Note: The crossbar moved out of this directory (2026-08-13). It is no longer an APB4-only block: `apbx_xbar` carries per-port version masks and can be APB4, APB5, or mixed on the same fabric, so it is documented in ../apbx/. An all-APB4 configuration — the default — is still built entirely from the APB4 primitives here, which is why the entry remains listed.

The RTL, generator, and testbenches live in the component area (`projects/components/fabric-gen-ip/lattice-apbx-xbar/`), not under `rtl/amba/apb4/`.

1.1.5 Clock-Gated Variants

Each adds `cfg_cg_enable` / `cfg_cg_idle_count` inputs and gating/idle status outputs. `apb4_master_cg` and `apb4_slave_cg` wrap their base module in one `amba_clock_gate_ctrl` with an `cg_gating` status output; `apb4_slave_cdc_cg` is a SIBLING of apb4_slave_cdc, not a

wrapper — it re-instantiates apb4_slave plus the two CDC FIFOs around TWO gate cells, with per-domain `pclk_cg_*` / `aclk_cg_*` status outputs.

Module	Base Module	Documentation	Status
apb4_master_cg	`apb4_master`	apb4_master_cg.md	Documented
apb4_slave_cg	`apb4_slave`	apb4_slave_cg.md	Documented
apb4_slave_cdc_cg	`apb4_slave_cdc`	apb4_slave_cdc_cg.md	Documented

1.1.6 Key Features

APB protocol support: - **Full APB4 Compliance:** Complete AMBA 4 APB protocol implementation - **PSTRB Support:** Byte-lane strobes for partial writes - **PPROT Support:** Protection attributes for security-aware systems - **Error Handling:** PSLVERR support for error responses - **APB5 Available Separately:** See `rtl/amba/apb5/` for `PWAKEUP`, the `P*USER` sidebands, and optional parity

Clock domain crossing: - **Dual-Clock Operation:** APB (pclk) and backend (aclk) domains - **Safe CDC:** Gray-pointer asynchronous FIFOs (`gaxi_fifo_async`) in both directions - **Independent Frequencies:** Backend can run faster or slower than APB - Caveat: **Reset both domains together** across the CDC variants — a one-sided reset is NOT safe (consumed entries replay / responses fabricate; see apb4_slave_cdc.md's reset analysis)

Monitoring and debug: - **Transaction Monitoring:** Real-time protocol monitoring - **128-bit Monitor Bus:** Standardized packet format plus a 64-bit side-band timestamp - **Error Detection:** Protocol violations, timeout detection - **Performance Tracking:** Transaction counting, latency measurement

Testbench integration: - **Packed Interfaces:** Simplified testbench connectivity - **Stub Modules:** Lightweight wrappers for CocoTB integration - **WaveDrom Support:** Automated waveform generation

---

1.2 Functional Description

1.2.1 APB Transfer Phases

APB uses a simple two-phase protocol:

1. **SETUP Phase:** PSEL asserted, PENABLE low
   - Master presents address, control signals, and write data (if write)
   - Slave prepares for transfer

2. **ACCESS Phase:** PSEL and PENABLE both asserted
   - Slave completes transfer and asserts PREADY when ready
   - For reads, slave presents PRDATA
   - Slave may assert PSLVERR to signal error

1.2.2 Signal Descriptions

Signal	Direction	Description
PCLK	Input	APB clock
PRESETn	Input	Active-low reset
PSEL	Master→Slave	Slave select
PENABLE	Master→Slave	Enable (ACCESS phase indicator)
PREADY	Slave→Master	Transfer complete
PADDR	Master→Slave	Address bus
PWRITE	Master→Slave	Write enable (1=write, 0=read)
PWDATA	Master→Slave	Write data
PSTRB	Master→Slave	Byte lane strobes
PPROT	Master→Slave	Protection attributes
PRDATA	Slave→Master	Read data
PSLVERR	Slave→Master	Error response

1.3 Usage Example

1.3.1 Using APB Master

```
apb4_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .CMD_DEPTH(2),
    .RSP_DEPTH(2)
) u_apb4_master (
    .pclk           (clk),
    .presetn        (resetn),

    // Command interface
    .cmd_valid      (cmd_valid),
```

```
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (3'b000),      // floating this drives Z into the
FIFO

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),

    // APB master interface
    .m_apb_PSEL     (psel),
    .m_apb_PENABLE  (penable),
    .m_apb_PREADY   (pready),
    .m_apb_PADDR    (paddr),
    .m_apb_PWRITE   (pwrite),
    .m_apb_PWDATA   (pwdata),
    .m_apb_PSTRB    (pstrb),
    .m_apb_PRDATA   (prdata),
    .m_apb_PSLVERR  (pslverr)
);
```

1.3.2 Using APB Slave

```
apb4_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb4_slave (
    .pclk           (clk),
    .presetn        (resetn),

    // APB slave interface
    .s_apb_PSEL     (psel),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready),
    .s_apb_PADDR    (paddr),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PRDATA   (prdata),
    .s_apb_PSLVERR  (pslverr),

    // Command interface
```

```
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),   // OUTPUT to the backend (do not
tie)

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

1.4 Design Notes

1.4.1 Command/Response Architecture

All APB modules use a command/response interface pattern for backend integration:

Command Interface (Master → Backend or APB → Backend): - `cmd_valid`, `cmd_ready` - Handshake signals - `cmd_pwrite` - Write/read direction - `cmd_paddr` - Transaction address - `cmd_pwdata` - Write data - `cmd_pstrb` - Byte strobes - `cmd_pprot` - Protection attributes

Response Interface (Backend → Master or Backend → APB): - `rsp_valid`, `rsp_ready` - Handshake signals - `rsp_prdata` - Read data - `rsp_pslverr` - Error flag

This separation buys you: - Clean clock domain crossing - Buffering and pipelining - Easy testbench integration - Backend processing flexibility

1.4.2 Monitor Bus Protocol

The APB monitor outputs standardized 128-bit `monitor_packet_t` records, paired with a 64-bit side-band timestamp:

```
[127:124] - Packet Type    (4 bits)
[123:109] - Reserved       (15 bits, forward-compat slack)
[108:105] - Protocol       (4 bits)
[104:97]  - Event Code     (8 bits)
[96:88]   - Channel ID     (9 bits)
[87:72]   - Agent ID       (16 bits)
[71:64]   - Unit ID        (8 bits)
[63:0]    - Event Data     (64 bits)
```

See apb4_monitor.md for detailed packet format.

1.5 Related Modules

- **APB5 Modules** - APB5 family (PWAKEUP, P*USER, parity)
- **AXI4 Modules** - Full AXI4 protocol components
- **AXIL4 Modules** - AXI4-Lite components
- **AXIS4 Modules** - AXI4-Stream components
- **GAXI Modules** - Generic AXI utilities

1.6 Testing

All APB modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all APB tests
pytest val/amba/test_apb*.py -v

# Run specific module tests
pytest val/amba/test_apb4_master.py -v
pytest val/amba/test_apb4_slave.py -v
pytest val/amba/test_apb4_slave_cdc.py -v
pytest val/amba/test_apb4_monitor.py -v

# Clock-gated CDC variant
pytest val/amba/test_apb4_slave_cdc_cg.py -v

# Generated crossbars (component area)
pytest projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/ -
v

# Run with waveform generation
env ENABLE_WAVEDROM=1 pytest val/amba/test_apb4_slave_wavedrom.py -v
```

1.6.1 WaveDrom Tests

Several modules have dedicated WaveDrom tests that generate detailed timing diagrams:

- `test_apb4_slave_wavedrom.py::test_comprehensive_apb4_slave` - APB slave protocol waveforms
- `test_apb4_slave_cdc.py::test_apb4_slave_cdc_wavedrom` - CDC timing diagrams

1.6.2 Test Coverage Gaps

The following APB4 modules have no dedicated test at `val/amba/`. They are exercised indirectly (stubs through the generated crossbars and the AXI4-to-APB bridge; the clock-gated wrappers through their base modules), and direct tests exist for the APB5 equivalents:

Module	Direct APB4 test	APB5 equivalent
`apb4_master_stub`	None	`test_apb5_master_stub.py`
`apb4_slave_stub`	None	`test_apb5_slave_stub.py`
`apb4_master_cg`	None	`test_apb5_master_cg.py`
`apb4_slave_cg`	None	`test_apb5_slave_cg.py`

Generated waveforms are stored in: - JSON format: `_wavedrom/` - SVG images: `_wavedrom_svg/`

1.7 References

1.7.1 Specifications

- ARM IHI 0024C – AMBA APB Protocol Specification, Version 2.0, Apr. 2010 (defines APB4: adds `PSTRB` and `PPROT`). This is the specification these modules implement: ch. 3 “Transfers” and ch. 4 “Operating States”. developer.arm.com/documentation/ihi0024/c
- ARM IHI 0024E – AMBA APB Protocol Specification, Issue E, Feb. 2023 (APB5: `PWAKEUP`, `P*USER` sidebands, parity; APB5 first appeared in Issue D, Apr. 2021). Applies to `rtl/amba/apb5/`, not to this family.

1.7.2 Papers and textbooks

- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy

`amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.

- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 "Basic and Advanced On-Chip Busses and Interconnects", section "ARM AMBA interconnect protocols suite" (APB bus protocol overview) – textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, Part 1 ch. 7 "Common On-Chip Bus Protocols" – APB and AXI in the on-chip bus family (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, Part 2 ch. 2 "Handshake Protocols" (valid-ready protocol and valid-ready slices) – textbook treatment of the skid buffer (secondary source).
- Dally, W.J., Towles, B. *Principles and Practices of Interconnection Networks*, Morgan Kaufmann, 2004, ch. 22 "Buses", §22.1 "Bus Basics" and §22.2 "Bus Arbitration" – textbook treatment of the shared-bus model (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`)

Naming caution: ARM's *document* version 2.0 (IHI 0024C) is the APB4 specification. The older APB3 protocol is IHI 0024B. Citing "APB Protocol Specification v2.0" without the IHI number is ambiguous.

1.7.3 Source Code

- RTL: `rtl/amba/apb4/`
- Tests: `val/amba/test_apb*.py`
- Framework: `bin/TBClasses/components/apb/`

1.7.4 Test Documentation

- APB Master Tests: `val/amba/test_apb4_master.py`
- APB Slave Tests: `val/amba/test_apb4_slave.py`
- APB CDC Tests: `val/amba/test_apb4_slave_cdc.py`
- APB Monitor Tests: `val/amba/test_apb4_monitor.py`
- APB Crossbar Tests: `projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests/`

Last Updated: 2026-07-19

1.8 Navigation

- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

2 apb4_master

2.1 Overview

The `apb4_master` is a full AMBA 4 APB (APB4) master with integrated command and response buffering. It gives you a high-performance interface for accessing APB slave devices behind a simple command/response handshake, and it supports every APB4 feature — write strobes, protection attributes, and error responses included.

Protocol scope: APB4 only. For `PWAKEUP`, the `P*USER` sidebands, or optional parity, use `apb5_master` from `rtl/amba/apb5/` — see the APB5 book.

2.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	APB protection signal width
CMD_DEPTH	int	6	Command buffer depth in **entries** (not a log2 exponent)
RSP_DEPTH	int	6	Response buffer depth in **entries** (not a log2 exponent)
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

CMD_DEPTH and RSP_DEPTH are literal entry counts. Both buffers are `gaxi_skid_buffer` instances, which store their payload in an unpacked array of `DEPTH` slots. The default of 6 means six entries, not sixty-four. Supported values are 2..8 inclusive.

2.3 Ports

```
module apb4_master #(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int PROT_WIDTH      = 3,
    parameter int CMD_DEPTH       = 6,
    parameter int RSP_DEPTH       = 6,
    parameter int STRB_WIDTH      = DATA_WIDTH / 8,
    // Short Parameters
```

```
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH,
    parameter int CPW = AW + DW + SW + PW + 1,  // Command packet
width
    parameter int RPW = DW + 1                  // Response packet
width
) (
    // Clock and Reset
    input  logic              pclk,
    input  logic              presetn,

    // APB Master Interface
    output logic              m_apb_PSEL,
    output logic              m_apb_PENABLE,
    output logic [AW-1:0]     m_apb_PADDR,
    output logic              m_apb_PWRITE,
    output logic [DW-1:0]     m_apb_PWDATA,
    output logic [SW-1:0]     m_apb_PSTRB,
    output logic [PW-1:0]     m_apb_PPROT,
    input  logic [DW-1:0]     m_apb_PRDATA,
    input  logic              m_apb_PSLVERR,
    input  logic              m_apb_PREADY,

    // Command Interface
    input  logic              cmd_valid,
    output logic              cmd_ready,
    input  logic              cmd_pwrite,
    input  logic [AW-1:0]     cmd_paddr,
    input  logic [DW-1:0]     cmd_pwdata,
    input  logic [SW-1:0]     cmd_pstrb,
    input  logic [PW-1:0]     cmd_pprot,

    // Response Interface
    output logic              rsp_valid,
    input  logic              rsp_ready,
    output logic [DW-1:0]     rsp_prdata,
    output logic              rsp_pslverr
);
```

2.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low

Port	Width	Direction	Description
			reset

2.3.2 APB Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	APB write/read indicator
m_apb_PWDATA	DATA_WIDTH	Output	APB write data
m_apb_PSTRB	STRB_WIDTH	Output	APB write strobes
m_apb_PPROT	PROT_WIDTH	Output	APB protection attributes
m_apb_PRDATA	DATA_WIDTH	Input	APB read data
m_apb_PSLVERR	1	Input	APB slave error
m_apb_PREADY	1	Input	APB ready

2.3.3 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command ready (FIFO not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	ADDR_WIDTH	Input	Command

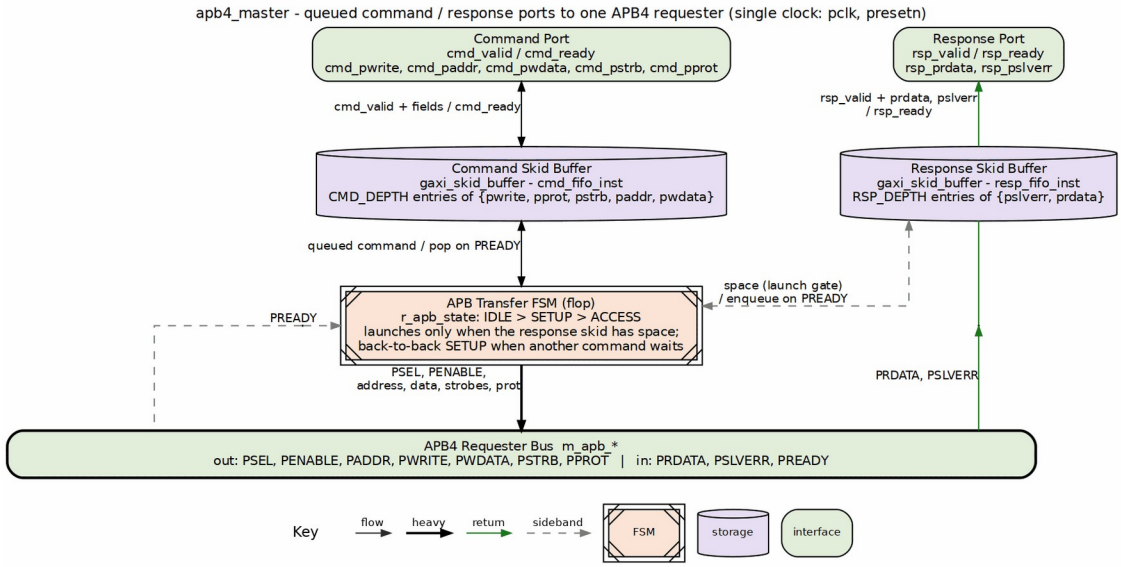
Port	Width	Direction	Description
			address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Command write strobes
cmd_pprot	PROT_WIDTH	Input	Command protection attributes

2.3.4 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DATA_WIDTH	Output	Response read data
rsp_pslverr	1	Output	Response error status

2.4 Functional Description

2.4.1 Block diagram



apb4_master block diagram

Commands drop down the left: the command port feeds the `CMD_DEPTH` command skid (`gaxi_skid_buffer`), and the transfer FSM pops one entry per transfer and drives PSEL, PENABLE and the address, data, strobe and protection fields onto the bus (IDLE > SETUP > ACCESS). PRDATA and PSLVERR climb the right lane into the `RSP_DEPTH` response skid and out the response port. The FSM launches a transfer only when the response skid has room, so a completed transfer can always be enqueued; PREADY, returned from the bus, completes ACCESS.

2.4.2 APB Protocol Implementation

The module implements the complete APB protocol state machine:

1. **IDLE**: No transaction active, waiting for commands
2. **SETUP**: PSEL asserted, transaction parameters set up
3. **ACCESS**: PENABLE asserted, waiting for PREADY from slave

2.4.3 Command Processing Flow

```
Command Interface → Command FIFO → APB State Machine → APB Interface
                                                              ↓
Response Interface ← Response FIFO ← APB Response Processing
```

2.4.4 Key Features

- **Buffered Operation**: Command and response buffers prevent blocking

- **APB4 Compliance**: Full protocol support including PSTRB and PPROT
- **Error Handling**: Proper PSLVERR propagation and handling
- **Flow Control**: Ready/valid handshaking on all interfaces

2.4.5 Protection Attributes (PPROT)

The module passes APB protection attributes through unmodified. Per the AMBA APB specification each bit is independent:

Bit	0	1
PPROT[0]	Normal access	Privileged access
PPROT[1]	Secure access	Non-secure access
PPROT[2]	Data access	Instruction access

Note the polarity of PPROT[1]: **0 means secure**, 1 means non-secure. The full encoding follows from the per-bit table:

PPROT[2:0]	Description
000	Normal, Secure, Data
001	Privileged, Secure, Data
010	Normal, Non-secure, Data
011	Privileged, Non-secure, Data
100	Normal, Secure, Instruction
101	Privileged, Secure, Instruction
110	Normal, Non-secure, Instruction
111	Privileged, Non-secure, Instruction

2.4.6 Write Strobe Support

Fine-grained byte-level write control:

```
// Example: Write only upper 16 bits of 32-bit register
cmd_pwdata <= {new_upper_data, 16'h0000};
cmd_pstrb  <= 4'b1100;  // Only upper 2 bytes
```

2.4.7 Error Handling

Comprehensive error handling and reporting:

```
always_ff @(posedge clk) begin
    if (rsp_valid && rsp_ready) begin
```

```
        if (rsp_pslverr) begin
            // Log error details
            error_count <= error_count + 1;
            error_addr  <= last_cmd_addr;
            error_type  <= last_cmd_write ? ERR_WRITE : ERR_READ;
        end
    end
end
```

2.5 Timing Characteristics

2.5.1 APB Transaction Timing

Characteristic	Value	Description
Setup Phase	2 clock cycles cold, 1 back-to-back	PSEL asserts in IDLE and again through SETUP before PENABLE; ACCESS -> SETUP skips the IDLE cycle
Access Phase	≥1 clock cycle	PENABLE assertion until PREADY
Total Latency	3+ clock cycles cold, 2+ back-to-back	Minimum transaction time (a standalone transfer pays the extra IDLE-entry cycle)
FIFO Latency	1 clock cycle	Command to APB delay
Response Latency	1 clock cycle	APB to response delay

2.5.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent, not characterized in this repository
Command Buffering	6 commands	With default `CMD_DEPTH=6`
Response Buffering	6 responses	With default `RSP_DEPTH=6`
Outstanding APB transfers	1	APB is non-pipelined

On throughput: APB is non-pipelined — the FSM issues one transfer at a time and cannot start the next until `PREADY` for the current one. Buffering lets the command producer run ahead and

lets responses drain lazily; it does not overlap bus transfers. Peak throughput is therefore `DATA_WIDTH / (transfer cycles × pclk period)`, with a minimum of two cycles (SETUP + one ACCESS cycle) per transfer against a zero-wait-state slave — back-to-back only; a cold (standalone) transfer takes THREE cycles, paying an extra PSEL cycle on the IDLE -> SETUP entry.

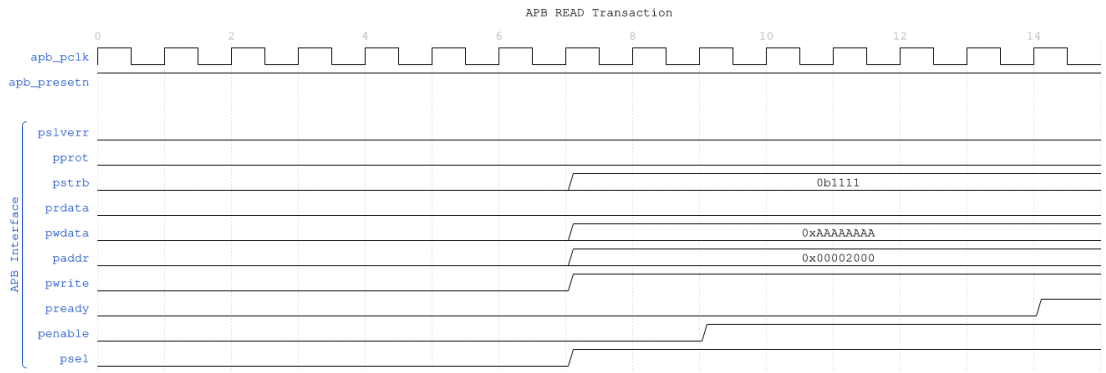
2.6 Waveforms

The following timing diagrams show comprehensive APB master behavior across 6 scenarios.

Known diagram defect: the WaveDrom generator stamps a fixed header string, `APB READ Transaction`, onto every scenario image regardless of direction. The signal traces themselves are correct — read `PWRITE` in the trace, not the header, to determine transaction direction.

2.6.1 Scenario 1: Basic Write Transaction

Shows APB write with command FIFO and response handling:

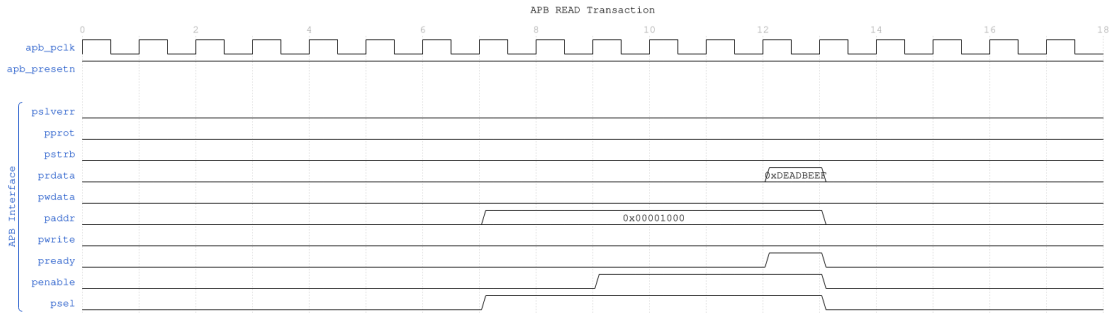


APB Write

WaveJSON: apb_write_sequence_001.json

Key Observations: - Command accepted into CMD FIFO - APB SETUP phase (PSEL=1) - APB ACCESS phase (PENABLE=1) - Response captured in RSP FIFO

2.6.2 Scenario 2: Basic Read Transaction

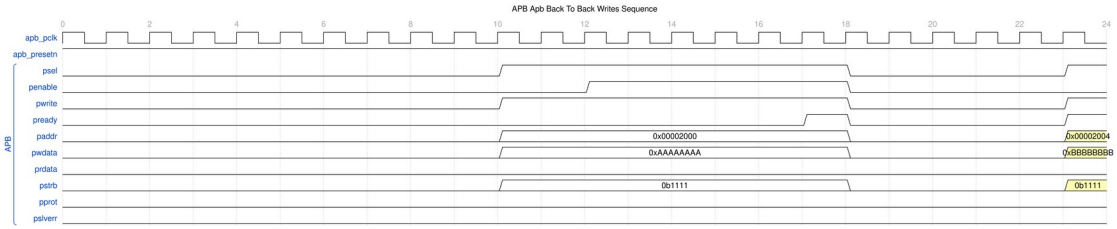


APB Read

WaveJSON: apb_read_sequence_001.json

Key Observations: - Read command with PWRITE=0 - APB protocol phases - Read data returned via response interface

2.6.3 Scenario 3: Back-to-Back Writes

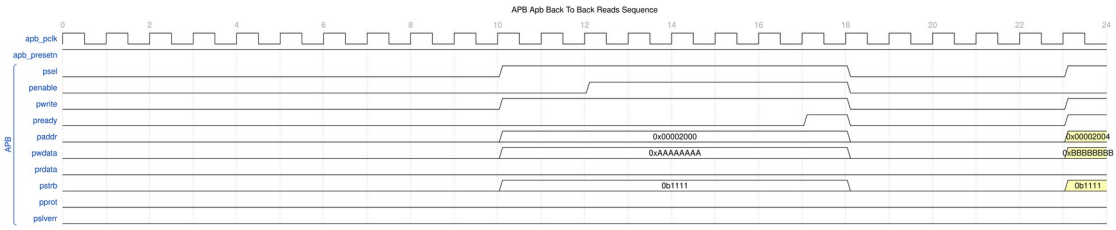


B2B Writes

WaveJSON: apb_back_to_back_writes_001.json

Key Observations: - Multiple commands queued in CMD FIFO - Sequential APB transactions - Pipelined command/response flow

2.6.4 Scenario 4: Back-to-Back Reads

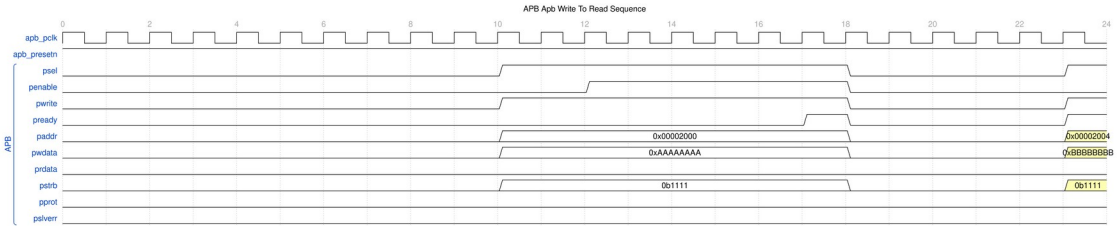


B2B Reads

WaveJSON: apb_back_to_back_reads_001.json

Key Observations: - Multiple read commands queued - Response FIFO accumulates read data - Burst read performance

2.6.5 Scenario 5: Write-to-Read Transition

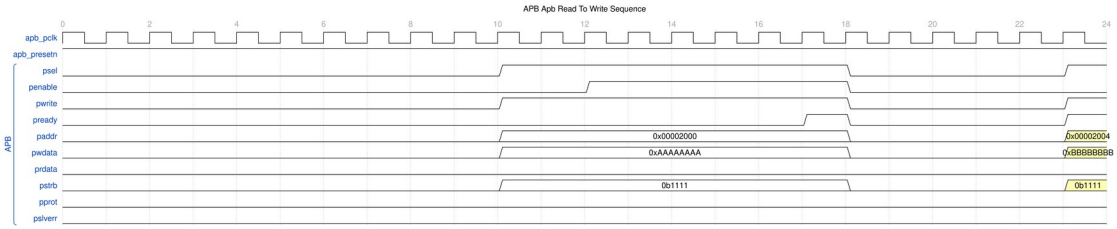


Write-to-Read

WaveJSON: apb_write_to_read_001.json

Key Observations: - Transition between write and read operations - No dead cycles between transaction types

2.6.6 Scenario 6: Read-to-Write Transition



Read-to-Write

WaveJSON: apb_read_to_write_001.json

Key Observations: - Seamless transition from read to write - Protocol state machine efficiency

---

2.7 Usage Examples

Every parameter and port below is read from the module declaration.

```
apb4_master #(
    .ADDR_WIDTH             (32),
    .DATA_WIDTH             (32),
    .PROT_WIDTH             (3),
    .CMD_DEPTH              (6),
    .RSP_DEPTH              (6)
) u_apb4_master (
    .pclk                   (pclk),
```

```
    .presetn                (presetn),
    .m_apb_PSEL             (m_apb_PSEL),
    .m_apb_PENABLE          (m_apb_PENABLE),
    .m_apb_PADDR            (m_apb_PADDR),
    .m_apb_PWRITE           (m_apb_PWRITE),
    .m_apb_PWDATA           (m_apb_PWDATA),
    .m_apb_PSTRB            (m_apb_PSTRB),
    .m_apb_PPROT            (m_apb_PPROT),
    .m_apb_PRDATA           (m_apb_PRDATA),
    .m_apb_PSLVERR          (m_apb_PSLVERR),
    .m_apb_PREADY           (m_apb_PREADY),
    .cmd_valid              (cmd_valid),
    .cmd_ready              (cmd_ready),
    .cmd_pwrite             (cmd_pwrite),
    .cmd_paddr              (cmd_paddr),
    .cmd_pwdata             (cmd_pwdata),
    .cmd_pstrb              (cmd_pstrb),
    .cmd_pprot              (cmd_pprot),
    .rsp_valid              (rsp_valid),
    .rsp_ready              (rsp_ready),
    .rsp_prdata             (rsp_prdata),
    .rsp_pslverr            (rsp_pslverr)
);
```

2.8 Design Notes

2.8.1 FIFO Depth Selection

Depths are entry counts and must be one of 2..8 inclusive. Because APB is non-pipelined, deeper buffers do not increase bus throughput — they only let the command producer run further ahead of the bus and absorb bursty response draining.

Application	CMD_DEPTH	RSP_DEPTH	Rationale
CPU Access	2	2	Low latency, one access at a time
DMA / block transfer	6	6	Producer runs ahead of the bus
Bridged (AXI4-to-APB)	8	8	Upstream pipelines commands well ahead of responses

2.8.2 Clock Gating

For power-sensitive applications, use `apb4_master_cg`. It wraps `apb4_master` in `amba_clock_gate_ctrl` and adds runtime configuration inputs rather than compile-time enables:

```
apb4_master_cg #(
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .CG_IDLE_COUNT_WIDTH (4)   // width of the idle countdown
) u_apb4_master_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    // Clock gating control (runtime inputs, not parameters)
    .cfg_cg_enable     (cg_enable),      // 1 = allow gating
    .cfg_cg_idle_count (4'd8),           // idle cycles before gating

    // ... all other ports identical to apb4_master ...

    // Status
    .cg_gating  (apb_cg_active)   // 1 while the clock is gated
);
```

See apb4_master_cg.md for the full interface.

2.8.3 Synthesis Considerations

Area: - Reduce FIFO depths for area-constrained designs - Use synchronous reset for smaller area - Consider shared resources for multiple APB masters

Timing: - Register all outputs for timing closure - Use appropriate FIFO depths to break critical paths - Consider skid buffers for high-frequency operation

Power: - Use clock gating variant when available - Implement power gating for inactive masters - Use appropriate FIFO depths to minimize switching

2.9 Related Modules

- **apb4_master_cg**: Clock-gated version for power optimization
- **apb4_master_stub**: Packed-interface variant wrapping this module, adds first/last framing
- **apb4_slave**: APB slave implementation
- **apb5_master**: APB5 equivalent with `PWAKEUP`, `P*USER` and optional parity
- **apbx_xbar_1to4 / apbx_xbar_2to4**: Generated APB crossbars for multi-slave systems

- **apb4_monitor**: APB protocol monitor and analyzer

2.10 Testing

2.10.1 Protocol Compliance

- Verify APB setup and access phases
- Check PSEL and PENABLE timing
- Validate PREADY handling and wait states

2.10.2 Interface Verification

- Test command/response FIFO overflow/underflow
- Verify backpressure handling
- Check error propagation and handling

2.10.3 Performance Verification

- Measure sustained throughput
- Verify FIFO efficiency
- Check latency under various loads

2.11 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c — ch. 4 “Operating States” (the IDLE, SETUP, ACCESS sequence this Requester drives) and ch. 3 “Transfers” (§3.1 write and §3.3 read transfers with `PREADY` wait states, §3.2 `PSTRB`, §3.4 `PSLVERR`, §3.5 `PPROT`). Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (APB bus protocol overview) — textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and

valid-ready slices) — textbook treatment of the skid buffer (secondary source).

- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

2.12 Navigation

- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

3 apb4_master_cg

Module: `apb4_master_cg.sv` **Base Module:** apb4_master **Location:** `rtl/amba/apb4/` **Status:** Production Ready

---

3.1 Overview

This is the **clock-gated variant** of apb4_master. It adds power optimization through activity-based clock gating, and it's a thin wrapper: an `amba_clock_gate_ctrl` instance produces `gated_pclk`, which feeds an otherwise unmodified `apb4_master`.

- **Same Functionality:** 100% equivalent to base module
- **Runtime Control:** Gating is enabled and tuned by input signals, not parameters
- **Observable:** `cg_gating` output reports when the clock is gated
- **Bypassable:** Tie `cfg_cg_enable = 0` for behaviour identical to the base module

For the clock-gating architecture and the underlying gate cell, see:

→ **Clock-Gated Variants Guide** and → **amba_clock_gate_ctrl**

Both live in the Shared book, which is published as a separate PDF. Take the parameter and port names for this module from the tables below, not from the generic guide.

3.2 Parameters

In addition to all apb4_master parameters:

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of the idle countdown counter; bounds the maximum programmable idle threshold

There is no `ENABLE_CLOCK_GATING` parameter and no per-domain `CG_GATE_*` parameters on this module — gating is controlled at runtime through the configuration inputs below.

3.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
STRB_WIDTH	DATA_WIDTH / 8
AW	ADDR_WIDTH
DW	DATA_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
ICW	CG_IDLE_COUNT_WIDTH
CPW	AW + DW + SW + PW + 1
RPW	DW + 1

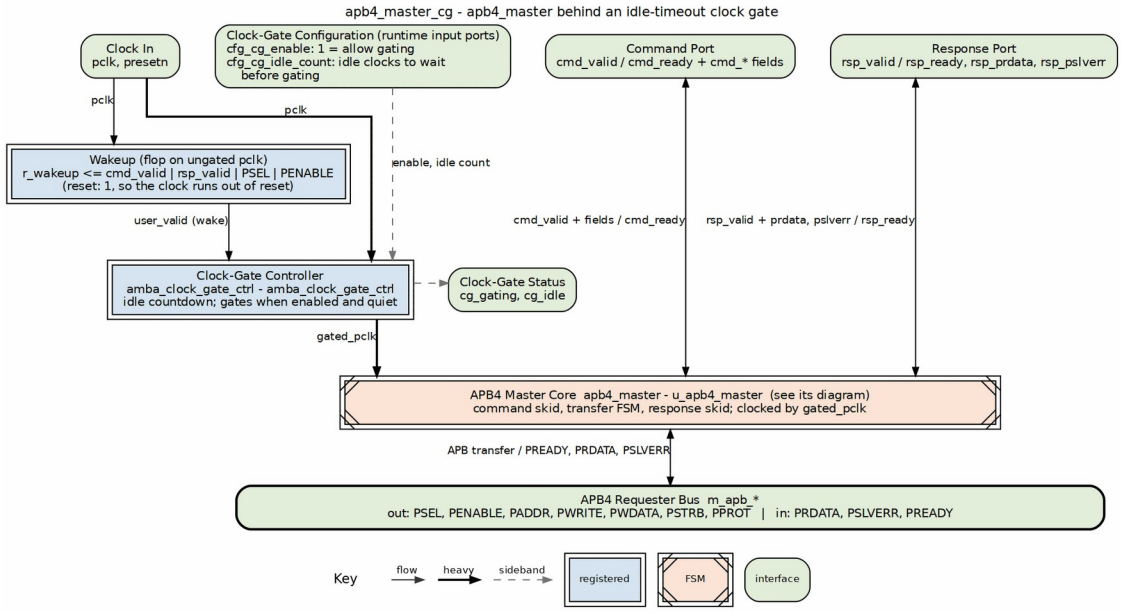
3.3 Ports

In addition to all apb4_master ports:

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Global clock-gate enable. 0 = never gate (identical to base module)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating the clock
cg_gating	1	Output	Asserted while the internal clock is gated
cg_idle	Out	1	Activity terms quiet (registered ~wakeup).

3.4 Functional Description

3.4.1 Block diagram



apb4_master_cg block diagram

The wrapped `apb4_master` is one box here; its own diagram shows the inside. What the variant adds is the clock path (heavy): `pclk` feeds `amba_clock_gate_ctrl`, whose gated clock runs the core. A wakeup flop on the ungated clock watches `cmd_valid`, `rsp_valid`, PSEL and PENABLE; when all have been quiet for `cfg_cg_idle_count` clocks and `cfg_cg_enable` is set, the clock stops. `cg_gating` and `cg_idle` report the state.

3.4.2 Wake-Up Condition

The wrapper holds the clock ungated while any of the following is true, sampled one cycle earlier into an internal `r_wakeup` register:

```
cmd_valid || rsp_valid || m_apb_PSEL || m_apb_PENABLE
```

That is: a pending command, a pending response, or an APB transfer in progress. Gating engages `cfg_cg_idle_count + 1` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after all four terms go low, because APB adds two register stages ahead of the ICG enable.

3.4.3 Wake-Up Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. This wrapper registers activity into its own `r_wakeup` flop and then hands it to `amba_clock_gate_ctrl`, which registers it again, so APB

is a **two-stage** family. The first gated-clock rising edge available to the wrapped `apb4_master` therefore arrives **3 cycles** after activity asserts.

3.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

3.6 Usage Examples

```
apb4_master_cg #(
    // Base module parameters (see apb4_master.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .CMD_DEPTH           (6),
    .RSP_DEPTH           (6),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    // Clock gating control
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),
    .cg_gating  (apb_cg_active),

    // ... all other ports same as apb4_master
);
```

Simulation note: set `cfg_cg_enable = 0` when debugging waveforms. A gated clock makes the internal state appear frozen and is easy to misread as a hang.

3.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

3.8 Related Modules

- **Base Module Functionality:** apb4_master.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

3.9 Testing

No test coverage. There is no `val/**/test_apb4_master_cg.py`, and no module that instantiates this one has a testbench either, so nothing in the repository exercises it.

Treat any behaviour described on this page as unverified by simulation.

3.10 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – ch. 3 "Transfers"

and ch. 4 “Operating States”, the protocol of the wrapped APB4 block. Citation only.

- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” – textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

3.11 Navigation

- **← Back to APB Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

4 apb4_master_stub

4.1 Overview

The `apb4_master_stub` is a lightweight APB master that accepts packed command packets and returns packed response packets. It exists for testbench use — when you need a simple APB master without the full functionality of the standard `apb4_master` module. The packed interface simplifies integration with test drivers and verification components.

4.2 Parameters

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command buffer depth in **entries** (not a log2 exponent)
RSP_DEPTH	int	6	Response buffer depth in **entries** (not a log2 exponent)
DATA_WIDTH	int	32	APB data bus width
ADDR_WIDTH	int	32	APB address bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
CMD_PACKET_WIDTH	int	Calculated	Command packet width
RESP_PACKET_WIDTH	int	Calculated	Response packet width

4.3 Ports

```
module apb4_master_stub #(
    parameter int CMD_DEPTH         = 6,
    parameter int RSP_DEPTH         = 6,
    parameter int DATA_WIDTH        = 32,
    parameter int ADDR_WIDTH        = 32,
    parameter int STRB_WIDTH        = DATA_WIDTH / 8,
    parameter int CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH +
STRB_WIDTH + 3 + 1 + 1 + 1,
                                      // addr, data, strb, prot,
pwrite, first, last
    parameter int RESP_PACKET_WIDTH = DATA_WIDTH + 1 + 1 + 1, // data,
pslverr, first, last
```

```
    // Short Parameters
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int CPW = CMD_PACKET_WIDTH,
    parameter int RPW = RESP_PACKET_WIDTH
) (
    // Clock and Reset
    input  logic                          pclk,
    input  logic                          presetn,

    // APB Master Interface
    output logic                          m_apb_PSEL,
    output logic                          m_apb_PENABLE,
    output logic [ADDR_WIDTH-1:0]         m_apb_PADDR,
    output logic                          m_apb_PWRITE,
    output logic [DATA_WIDTH-1:0]         m_apb_PWDATA,
    output logic [STRB_WIDTH-1:0]         m_apb_PSTRB,
    output logic [2:0]                    m_apb_PPROT,
    input  logic [DATA_WIDTH-1:0]         m_apb_PRDATA,
    input  logic                          m_apb_PSLVERR,
    input  logic                          m_apb_PREADY,

    // Command Packet Interface (packed)
    input  logic                          cmd_valid,
    output logic                          cmd_ready,
    input  logic [CMD_PACKET_WIDTH-1:0]   cmd_data,

    // Response Packet Interface (packed)
    output logic                          rsp_valid,
    input  logic                          rsp_ready,
    output logic [RESP_PACKET_WIDTH-1:0] rsp_data
);
```

4.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

4.3.2 APB Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	Peripheral select

Port	Width	Direction	Description
m_apb_PENABLE	1	Output	Enable signal
m_apb_PADDR	AW	Output	Address bus
m_apb_PWRITE	1	Output	Write/read (1=write, 0=read)
m_apb_PWDATA	DW	Output	Write data
m_apb_PSTRB	SW	Output	Write strobe
m_apb_PPROT	3	Output	Protection attributes
m_apb_PRDATA	DW	Input	Read data
m_apb_PSLVERR	1	Input	Slave error
m_apb_PREADY	1	Input	Ready signal

4.3.3 Packed Command Interface

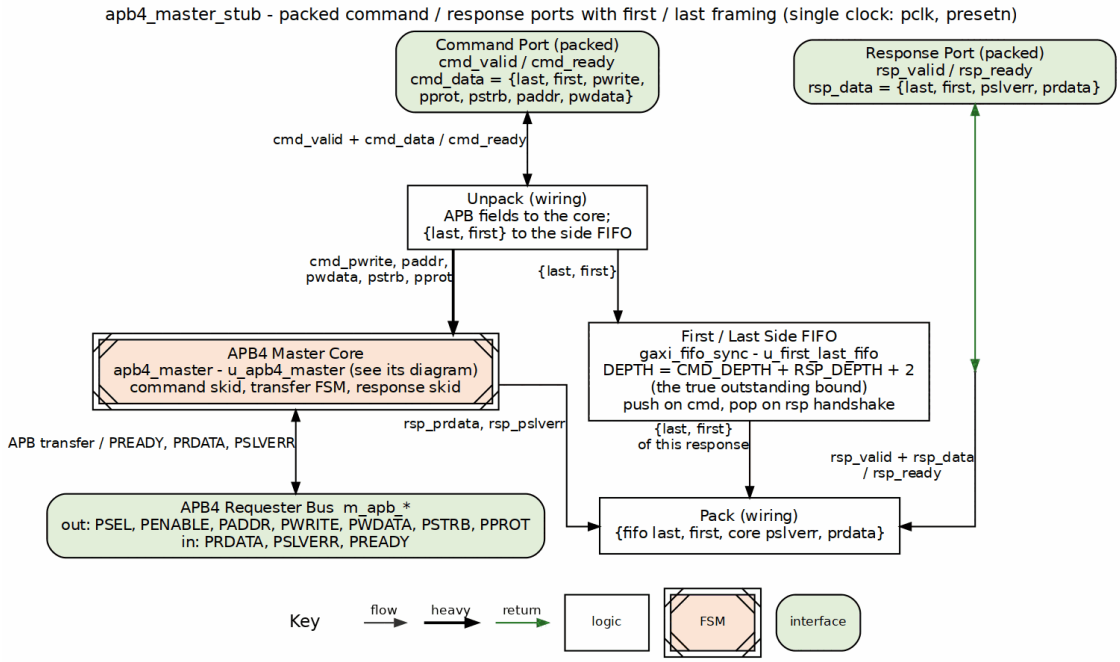
Port	Width	Direction	Description
cmd_valid	1	Input	Command packet valid
cmd_ready	1	Output	Ready for command packet
cmd_data	CPW	Input	Packed command (addr, data, strb, prot, pwrite, first, last)

4.3.4 Packed Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response packet valid
rsp_ready	1	Input	Ready for response packet
rsp_data	RPW	Output	Packed response (data, pslverr, first, last)

4.4 Functional Description

4.4.1 Block diagram



apb4_master_stub block diagram

A packed-port wrapper around `apb4_master` (one box; see its diagram). The packed command is unpacked: the APB fields go to the core, and its `{last, first}` framing bits go into a side FIFO (`gaxi_fifo_sync`, depth `CMD_DEPTH + RSP_DEPTH + 2`, the true outstanding bound). When the core returns a response, the side FIFO pops the framing of that same command and Pack builds `rsp_data`, which climbs the right edge to the response port. This pairs each response with its own command even when the next command is already presented.

4.4.2 Packet Interface

The stub uses packed interfaces to simplify testbench integration:

Command Packet Format (MSB to LSB): - `last` (1 bit): Last transfer indicator - `first` (1 bit): First transfer indicator - `pwrite` (1 bit): Write/read operation - `pprot` (3 bits): Protection attributes - `pstrb` (SW bits): Write strobe - `paddr` (AW bits): Address - `pwdata` (DW bits): Write data — the LSB field (paddr sits ABOVE pwdata)

Response Packet Format (MSB to LSB): - `last` (1 bit): Last transfer indicator - `first` (1 bit): First transfer indicator - `pslverr` (1 bit): Slave error - `prdata` (DW bits): Read data

4.4.3 First/Last Framing

`first` and `last` exist so that AXI4 burst framing survives the trip through an APB bridge. `apb4_master` does not carry them through its own command-to-response pipeline, so the stub keeps a small side FIFO (`gaxi_fifo_sync`, depth `CMD_DEPTH`): `{last, first}` is enqueued on every command-side handshake and dequeued on every response-side handshake.

This pairing is essential when the upstream pipelines commands ahead of responses. Tapping `first`/`last` combinationally from the live `cmd_data` input instead — as an earlier revision did — pairs command N's response with command N+1's framing bits as soon as the internal command FIFO accepts N+1. In the AXI4-to-APB bridge that manifested as `axi4_to_apb4_convert` never seeing `first=1` again in `RSP_IDLE`, hanging the FSM and surfacing as a timeout on the master AXI4 R channel.

The side FIFO drains on the EXTERNAL response handshake, so under response backpressure the accepted-not-yet-drained count can reach CMD_DEPTH + 1 + RSP_DEPTH — and the FIFO is sized for exactly that bound (`CMD_DEPTH + RSP_DEPTH + 2`), with a loud simulation `$error` tripwire should a future change break it. (The original CMD_DEPTH sizing silently dropped framing records under backpressure; that was TASK-067, fixed.)

4.4.4 Packet Format Is Not Symmetric With `apb4_slave_stub`

`apb4_slave_stub` omits `first` and `last` in both directions and is therefore two bits narrower per packet. The two stubs face each other across the APB bus, never packed-side to packed-side, so this asymmetry is intentional and causes no integration problem — but the packed buses must not be wired together directly. See apb4_slave_stub.md for the side-by-side comparison.

4.4.5 Operation

1. Test driver presents packed command on `cmd_data` with `cmd_valid=1`
2. Stub accepts when `cmd_ready=1`
3. Stub unpacks command, enqueues `{last, first}`, and drives APB protocol
4. On APB completion, stub packs the response with the dequeued `{last, first}` and asserts `rsp_valid=1`
5. Test driver reads response when `rsp_ready=1`

4.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

4.6 Usage Examples

```
// Instantiate APB master stub
apb4_master_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(16)
) u_apb4_master_stub (
    .pclk(clk),
    .presetn(rst_n),
    // APB interface to slave/interconnect
    .m_apb_PSEL(apb_psel),
    .m_apb_PENABLE(apb_penable),
    .m_apb_PADDR(apb_paddr),
    .m_apb_PWRITE(apb_pwrite),
    .m_apb_PWDATA(apb_pwdata),
    .m_apb_PSTRB(apb_pstrb),
    .m_apb_PPROT(apb_pprot),
    .m_apb_PRDATA(apb_prdata),
    .m_apb_PSLVERR(apb_pslverr),
    .m_apb_PREADY(apb_pready),
    // Packed interface to test driver
    .cmd_valid(test_cmd_valid),
    .cmd_ready(test_cmd_ready),
    .cmd_data(test_cmd_data),
    .rsp_valid(test_rsp_valid),
    .rsp_ready(test_rsp_ready),
    .rsp_data(test_rsp_data)
);

// Example: Send write command (in testbench)
// Pack command: addr=0x1000, data=0xDEADBEEF, write=1
// Field order per the RTL unpack: {last, first, pwrite, pprot, pstrb,
paddr, pwdata}
assign test_cmd_data = {1'b1, 1'b1, 1'b1, 3'b000, 4'hF, 16'h1000,
32'hDEADBEEF};
assign test_cmd_valid = 1'b1;
```

4.7 Design Notes

4.7.1 Testbench Usage

This module is intended for: - System-level testbenches - Integration testing - Simple APB traffic generation - Verification component integration

For production use, consider the full-featured `apb4_master` module.

4.7.2 Packed Interface Benefits

- Simplified connection to test drivers
- Reduces signal count
- Easier integration with verification IP
- Convenient for parameterized test sequences

4.8 Related Modules

- `apb4_master.sv` - Full-featured APB master with independent interfaces
- `apb4_slave_stub.sv` - Companion APB slave stub
- `apb4_monitor.sv` - APB transaction monitoring

4.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

4.10 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – ch. 3 “Transfers” and ch. 4 “Operating States”, the protocol of the wrapped APB4 block. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a

buffered stage may stall or add latency without changing what the endpoints compute.

- **Full APB Master**: apb4_master.md
- **APB Slave Stub**: apb4_slave_stub.md
- **APB5 Equivalent**: apb5_master_stub.md
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

4.11 Navigation

- **← Back to APB Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

5 apb4_monitor

5.1 Overview

The `apb4_monitor` is an APB transaction monitor with comprehensive error detection, performance tracking, and debug capabilities, all reported through a standardized monitor bus interface. It attaches to APB master cmd/rsp interfaces (timing-convenient proxies for APB signals) and generates standardized event packets — 128-bit monitor bus protocol, paired with a 64-bit side-band timestamp — for error detection, latency analysis, and transaction debugging.

5.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
UNIT_ID	logic [7:0]	8'h01	8-bit Unit identifier for monitor packets
AGENT_ID	logic [15:0]	16'h000A	16-bit Agent identifier for monitor packets
MAX_TRANSACTIONS	int	4	Maximum concurrent transactions (APB typically 1-4)
N_ADDR_RANGES	int	0	Address-range checker ranges; 0 disables the checker
MONITOR_FIFO_DEPTH	int	8	Internal FIFO depth for monitor packets
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.

5.3 Ports

```
module apb4_monitor
    import ocelus_common_pkg::*;  // PROTOCOL_APB, PktType*,
transaction states
```

```
    import ocelus_amba4_pkg::*;   // APB_ERR_*, APB_TIMEOUT_*, etc.
    // (`import ocelus_pkg::*;` is intentionally omitted -- its
helpers
    // duplicate ocelus_common_pkg's and Vivado flags the wildcard
overlap)
#(
    parameter bit USE_MONITOR         = 1'b1,  // 0 = omit monitor
body, tie outputs
    parameter int N_ADDR_RANGES       = 0,     // 0 = address-range
checker disabled
    parameter int ADDR_WIDTH          = 32,
    parameter int DATA_WIDTH          = 32,
    parameter logic [7:0]  UNIT_ID    = 8'h01,     // 8-bit Unit ID
    parameter logic [15:0] AGENT_ID   = 16'h000A,  // 16-bit Agent ID
    parameter int MAX_TRANSACTIONS    = 4,     // APB is typically
single outstanding
    parameter int MONITOR_FIFO_DEPTH  = 8,     // Monitor packet FIFO
depth

    // Short params
    parameter int AW                  = ADDR_WIDTH,
    parameter int DW                  = DATA_WIDTH,
    parameter int SW                  = DW/8
)
(
    // Clock and Reset (aclk domain - matches cmd/rsp interfaces)
    input  logic                      aclk,
    input  logic                      aresetn,

    // Command Interface Monitoring (aclk domain)
    input  logic                      cmd_valid,
    input  logic                      cmd_ready,
    input  logic                      cmd_pwrite,
    input  logic [AW-1:0]             cmd_paddr,
    input  logic [DW-1:0]             cmd_pwdata,
    input  logic [SW-1:0]             cmd_pstrb,
    input  logic [2:0]                cmd_pprot,

    // Response Interface Monitoring (aclk domain)
    input  logic                      rsp_valid,
    input  logic                      rsp_ready,
    input  logic [DW-1:0]             rsp_prdata,
    input  logic                      rsp_pslverr,

    // Configuration - Error Detection
    input  logic                      cfg_error_enable,        //
Enable error event packets
```

```
    input  logic                    cfg_timeout_enable,      //
Enable timeout event packets
    input  logic                    cfg_protocol_enable,     //
Enable protocol violation detection
    input  logic                    cfg_slverr_enable,       //
Enable slave error detection

    // Configuration - Performance Monitoring
    input  logic                    cfg_perf_enable,         //
Enable performance packets
    input  logic                    cfg_latency_enable,      //
Enable latency tracking
    input  logic                    cfg_throughput_enable,   //
ACCEPTED BUT UNIMPLEMENTED -- no throughput event exists (see Design
Notes)

    // Configuration - Debug
    input  logic                    cfg_debug_enable,        //
Enable debug packets
    input  logic                    cfg_trans_debug_enable,  //
Enable transaction debug
    input  logic [3:0]              cfg_debug_level,         //
ACCEPTED BUT UNIMPLEMENTED -- never referenced in the body

    // Configuration - Thresholds and Timeouts
    input  logic [15:0]             cfg_cmd_timeout_cnt,     //
Command timeout (cycles)
    input  logic [15:0]             cfg_rsp_timeout_cnt,     //
Response timeout (cycles)
    input  logic [31:0]             cfg_latency_threshold,   //
Latency threshold (cycles)
    input  logic [15:0]             cfg_throughput_threshold, //
ACCEPTED BUT UNIMPLEMENTED

    // Address-range checker (elaborated when N_ADDR_RANGES > 0)
    input  logic
cfg_addr_check_enable,
    input  logic [(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]
cfg_addr_range_enable,
    input  logic [(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0][AW-1:0]
cfg_addr_range_low,
    input  logic [(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0][AW-1:0]
cfg_addr_range_high,

    // Consolidated 128-bit event packet interface (monitor bus) + 64-
bit side-band timestamp
    output logic                    monbus_valid,            //
```

```
Monitor bus valid
    input  logic                    monbus_ready,           //
Monitor bus ready
    output logic [127:0]            monbus_packet,          //
Consolidated monitor packet (monitor_packet_t)
    output logic [63:0]             monbus_timestamp,       // Side-
band timestamp (monbus_timestamp_t)
    input  logic [63:0]             i_mon_time,             // Free-
running counter from monbus_group_core

    // Status outputs
    output logic [7:0]              active_count,           //
Number of active transactions
    output logic [15:0]             error_count,            // Total
error count
    output logic [31:0]             transaction_count       // Total
transaction count
);
```

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	Monitor clock (matches cmd/rsp domain)
aresetn	1	Input	Active-low asynchronous reset

5.3.2 Command Interface Monitoring

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid signal
cmd_ready	1	Input	Command ready signal
cmd_pwrite	1	Input	Write/read indicator (1=write, 0=read)
cmd_paddr	AW	Input	Command

Port	Width	Direction	Description
			address
cmd_pwdata	DW	Input	Write data
cmd_pstrb	SW	Input	Write strobe
cmd_pprot	3	Input	Protection attributes

5.3.3 Response Interface Monitoring

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid signal
rsp_ready	1	Input	Response ready signal
rsp_prdata	DW	Input	Read data
rsp_pslverr	1	Input	Slave error indicator

5.3.4 Configuration - Error Detection

Port	Width	Direction	Description
cfg_error_enable	1	Input	Enable error event packet generation
cfg_timeout_enable	1	Input	Enable timeout detection
cfg_protocol_enable	1	Input	Enable protocol violation detection
cfg_slverr_enable	1	Input	Enable slave error reporting

5.3.5 Configuration - Performance Monitoring

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance packet generation
cfg_latency_enable	1	Input	Enable latency measurement
cfg_throug	1	Input	ACCEPTED BUT

Port	Width	Direction	Description
hput_enable			UNIMPLEMENTED: never referenced in the module body; no throughput event exists

5.3.6 Configuration - Debug

Port	Width	Direction	Description
cfg_debug_enable	1	Input	Enable debug packet generation
cfg_trans_debug_enable	1	Input	Enable transaction-level debugging
cfg_debug_level	4	Input	ACCEPTED BUT UNIMPLEMENTED: never referenced in the module body

5.3.7 Configuration - Thresholds

Port	Width	Direction	Description
cfg_cmd_timeout_cnt	16	Input	Command timeout threshold (clock cycles)
cfg_rsp_timeout_cnt	16	Input	Response timeout threshold (clock cycles)
cfg_latency_threshold	32	Input	Latency threshold for alerts (clock cycles)
cfg_throughput_threshold	16	Input	ACCEPTED BUT UNIMPLEMENTED: never referenced in the module body
cfg_addr_check_enable	1	Input	Enable the address-range checker (needs N_ADDR_RANGES > 0)
cfg_addr_range_enable	N_ADDR_RANGES	Input	Per-range enable

Port	Width	Direction	Description
cfg_addr_range_low	N_ADDR_RANGES x AW	Input	Per-range low bound
cfg_addr_range_high	N_ADDR_RANGES x AW	Input	Per-range high bound

5.3.8 Monitor Bus Interface

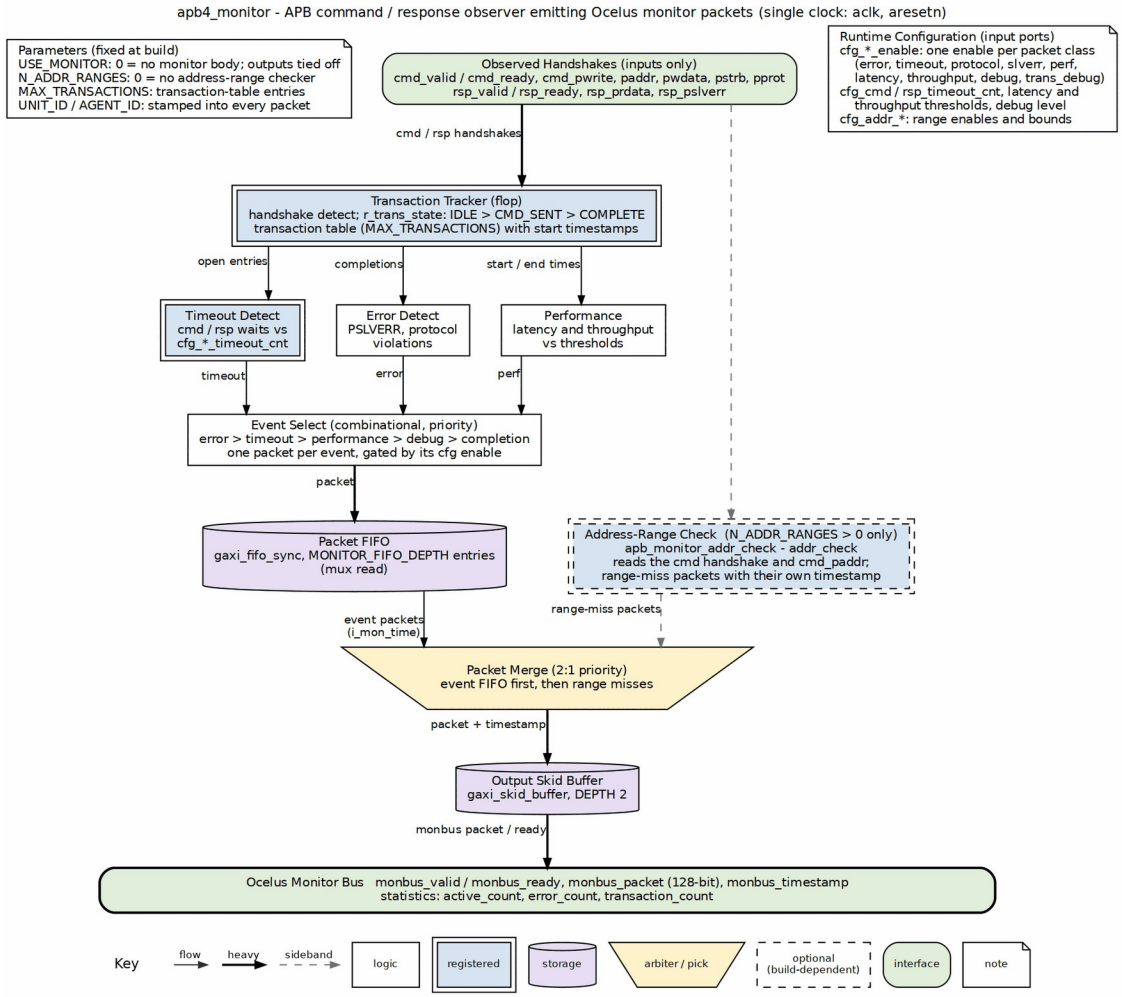
Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	Consolidated `monitor_packet_t` (see format below)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

5.3.9 Status Outputs

Port	Width	Direction	Description
active_count	8	Output	Number of currently active transactions
error_count	16	Output	Cumulative error count
transaction_count	32	Output	Total transaction count

5.4 Functional Description

5.4.1 Block diagram



apb4_monitor block diagram

A passive observer of the command / response handshakes. The transaction tracker follows each transfer (IDLE > CMD_SENT > COMPLETE) in a `MAX_TRANSACTIONS`-entry table with start timestamps. Timeout, error and performance checks run in parallel and feed the event select, which emits one packet per event in priority order (error, timeout, performance, debug, completion), each gated by its `cfg_*_enable`; packets queue in a `MONITOR_FIFO_DEPTH` FIFO. With `N_ADDR_RANGES > 0`, the address-range checker watches the command handshake directly and produces its own range-miss packets; a 2:1 priority merge (event FIFO first) chooses between the two streams, and a 2-entry skid buffer drives the Ocelus monitor bus.

5.4.2 Transaction Monitoring

The APB monitor tracks transactions through the command/response pipeline:

1. **Command Phase**: Monitors cmd_valid/cmd_ready handshake
   - Captures address, write/read type, data, strobe
   - Starts latency counter
   - Detects protocol violations
2. **Response Phase**: Monitors rsp_valid/rsp_ready handshake
   - Captures response data and error status
   - Calculates transaction latency
   - Matches response to command

5.4.3 Event Detection

The monitor generates standardized 128-bit packets (paired with 64-bit side-band timestamps) for:

Error Events (when cfg_error_enable = 1): - SLVERR responses (when cfg_slverr_enable = 1) - Protocol violations (when cfg_protocol_enable = 1) - Timeout conditions (when cfg_timeout_enable = 1)

Performance Events (when cfg_perf_enable = 1): - Latency threshold violations (`cfg_latency_threshold` compare — the only perf event the RTL generates)

Debug Events (when cfg_debug_enable = 1): - Transaction start/completion - State transitions - Internal status changes

5.4.4 Monitor Packet Format

The 128-bit `monbus_packet` (paired with the 64-bit `monbus_timestamp` side-band signal) follows the standardized APB monitor bus format. The layout is identical across protocols:

```
Bits [127:124] - Packet Type:
  0x0 = ERROR      Error events (SLVERR, protocol violations)
  0x1 = COMPL      Completion events (transaction finished). NOTE:
completion
                   packets are UNGATED -- one monbus packet per
successful
                   transaction in EVERY configuration; there is no
                   cfg_compl_enable on this monitor. Budget the
consumer for it.
  0x2 = THRESH     Threshold events
  0x3 = TIMEOUT    Timeout events
  0x4 = PERF       Performance metrics
  0x8 = ADDR_MATCH Address match events
  0x9 = APB        APB-specific events
  0xF = DEBUG      Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB,
```

```
0x3=ARB, 0x4=CORE
Bits [104:97]  - Event Code (8 bits, protocol-specific)
Bits [96:88]   - Channel ID (9 bits)
Bits [87:72]   - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64]   - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0]    - Event Data (64 bits — full address, latency, etc.)
```

5.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

5.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
apb4_monitor #(
    .USE_MONITOR           (1'b1),
    .N_ADDR_RANGES         (0),
    .ADDR_WIDTH            (32),
    .DATA_WIDTH            (32),
    .UNIT_ID               (8'h01),
    .AGENT_ID              (16'h000A),
    .MAX_TRANSACTIONS      (4),
    .MONITOR_FIFO_DEPTH    (8)
) u_apb4_monitor (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cmd_valid             (cmd_valid),
    .cmd_ready             (cmd_ready),
    .cmd_pwrite            (cmd_pwrite),
    .cmd_paddr             (cmd_paddr),
    .cmd_pwdata            (cmd_pwdata),
    .cmd_pstrb             (cmd_pstrb),
    .cmd_pprot             (cmd_pprot),
    .rsp_valid             (rsp_valid),
    .rsp_ready             (rsp_ready),
    .rsp_prdata            (rsp_prdata),
```

```
    .rsp_pslverr           (rsp_pslverr),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_protocol_enable   (cfg_protocol_enable),
    .cfg_slverr_enable     (cfg_slverr_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_latency_enable    (cfg_latency_enable),
    .cfg_throughput_enable (cfg_throughput_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_trans_debug_enable(cfg_trans_debug_enable),
    .cfg_debug_level       (cfg_debug_level),
    .cfg_cmd_timeout_cnt   (cfg_cmd_timeout_cnt),
    .cfg_rsp_timeout_cnt   (cfg_rsp_timeout_cnt),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_throughput_threshold(cfg_throughput_threshold),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .active_count          (active_count),
    .error_count           (error_count),
    .transaction_count     (transaction_count)
);
```

5.7 Design Notes

5.7.1 Transaction Tracking

- APB is typically single-outstanding, so MAX_TRANSACTIONS=4 is usually sufficient
- Monitor tracks cmd→rsp matching to detect orphaned responses
- Separate timeout counters for command and response phases

5.7.2 Packet Generation

- Internal FIFO buffers monitor packets (depth = MONITOR_FIFO_DEPTH)
- Backpressure on `monbus_ready` stops at the internal FIFO: packet generation fires purely from event conditions and does NOT consult the FIFO's ready — a full FIFO silently DROPS the packet (lossy-but-honest; size MONITOR_FIFO_DEPTH and drain promptly if you need every event)

- The slot is freed either way: terminal transaction-table entries retire unconditionally, packet delivered or not (the historical leak-until-wedged behavior was TASK-066, fixed and witnessed by `apb4_monitor_slot_retire_test`)

5.7.3 Performance Considerations

- Disable unused packet types to reduce traffic
- Performance mode (cfg_perf_enable) can generate high packet rates
- Debug mode should only be used for targeted debugging

5.8 Related Modules

- `apb4_master.sv` - APB master with cmd/rsp interfaces
- `apb4_slave.sv` - APB slave with cmd/rsp interfaces
- `ocelus_common_pkg.sv` / `ocelus_amba4_pkg.sv` - Monitor packet definitions and APB event codes
- `gaxi_fifo_sync.sv` - Recommended for monitor packet buffering

5.9 Testing

`val/amba/test_apb4_monitor.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb4_monitor.py -v
```

5.10 References

- **Monitor Packet Specification**: monitor_package_spec.md
- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – ch. 3 “Transfers” (§3.4 “Error response”: the `PSLVERR` the monitor reports as error events) and ch. 4 “Operating States” (the SETUP/ACCESS sequence its command and response timeouts measure); the timeouts themselves are repo monitoring policy with no spec or paper behind them. Citation only.
- **Verification Guide**: See testbench in `val/amba/test_apb4_monitor.py`
- Indexed in `rtl/amba/references/README.md`

5.11 Navigation

- ← Back to APB4 Book
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

6 apb4_slave

6.1 Overview

The apb4_slave is a complete AMBA 4 APB (APB4) compliant slave with integrated command and response buffering built on GAXI skid buffers. It translates between the APB interface and a simple command/response interface, so dropping a custom peripheral or register block onto an APB-based system is straightforward.

Protocol scope: APB4 only. For PWAKEUP, the P*USER sidebands, or optional parity, use apb5_slave from rtl/amba/apb5/ — see the APB5 book.

6.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	APB protection signal width
DEPTH	int	2	Skid buffer depth in **entries** (not a log2 exponent)

DEPTH is a literal entry count. gaxi_skid_buffer stores its payload in an unpacked array of DEPTH slots, so DEPTH=2 gives two entries, not four. Supported values are 2..8 inclusive — ANY integer in that range (odd depths are legal); the shift-register storage is optimal at 2 and remains cheaper than a packed-vector implementation through 8. Values outside 2..8 fail elaboration.

6.3 Ports

```
module apb4_slave #(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int STRB_WIDTH      = DATA_WIDTH / 8,
    parameter int PROT_WIDTH      = 3,
    parameter int DEPTH           = 2,
    // Short Parameters
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH,
```

```
    parameter int CPW = AW + DW + SW + PW + 1,  // Command packet
width
    parameter int RPW = DW + 1                  // Response packet
width
) (
    // Clock and Reset
    input  logic              pclk,
    input  logic              presetn,

    // APB Slave Interface
    input  logic              s_apb_PSEL,
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

    // Command Interface
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

6.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

6.3.2 APB Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	APB ready signal
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	APB write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	APB write data
s_apb_PSTRB	STRB_WIDTH	Input	APB write strobes
s_apb_PPROT	PROT_WIDTH	Input	APB protection attributes
s_apb_PRDATA	DATA_WIDTH	Output	APB read data
s_apb_PSLVERR	1	Output	APB slave error

6.3.3 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid
cmd_ready	1	Input	Command ready (from backend)
cmd_pwrite	1	Output	Command write/read
cmd_paddr	ADDR_WIDTH	Output	Command address
cmd_pwdata	DATA_WIDTH	Output	Command write data

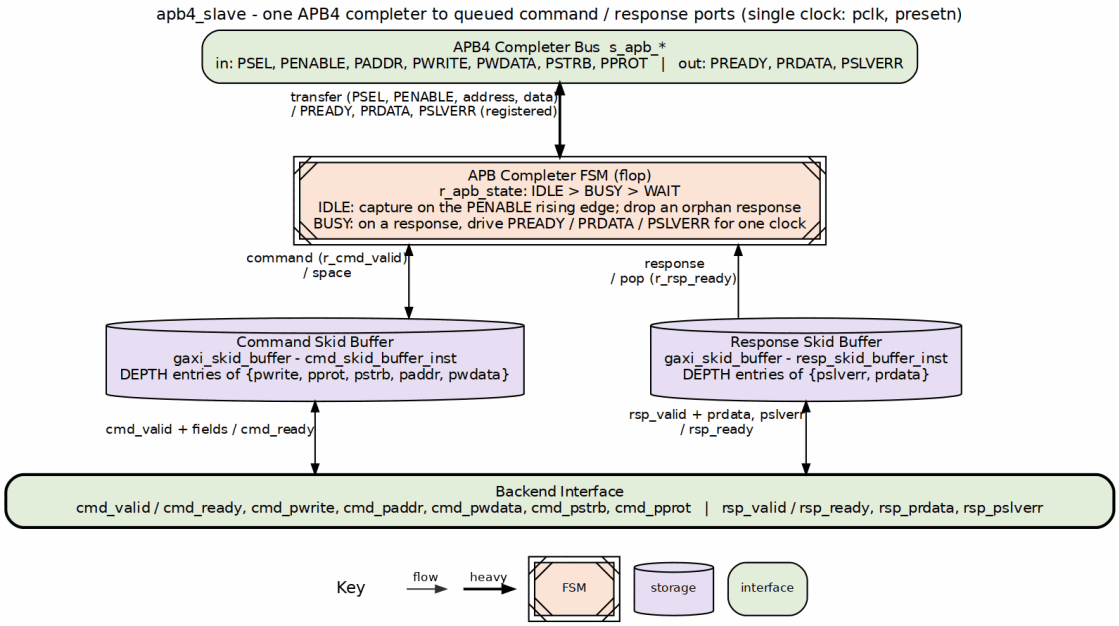
Port	Width	Direction	Description
cmd_pstrb	STRB_WIDTH	Output	Command write strobes
cmd_pprot	PROT_WIDTH	Output	Command protection attributes

6.3.4 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid
rsp_ready	1	Output	Response ready
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Response error status

6.4 Functional Description

6.4.1 Block diagram



apb4_slave block diagram

The APB bus enters at the top. The completer FSM (IDLE > BUSY > WAIT) captures one command on the rising edge of PENABLE and pushes it down the left through the command skid to the backend. The backend's response climbs the right lane through the response skid; in BUSY the FSM pops it and drives PREADY, PRDATA and PSLVERR for one clock. A response that arrives with nothing outstanding is popped and dropped in IDLE (the orphan-response guard).

6.4.2 APB Protocol Implementation

The module implements the complete APB slave protocol with a three-state finite state machine.

These are internal slave states, not APB bus phases. The APB SETUP and ACCESS phases are driven by the *master* (`PSEL` / `PENABLE`); a slave only observes them. `IDLE`, `BUSY`, and `WAIT` describe where this slave is in servicing the observed transfer:

1. **IDLE**: No transfer in flight. Waits for the SETUP-to-ACCESS edge (the cycle in which `PENABLE` first goes high while `PSEL` is high). Also drops any orphan response — see below.
2. **BUSY**: Command has been pushed to the backend; the bus is in its ACCESS phase with `PREADY` low (wait states). Waits for the backend response.
3. **WAIT**: `PREADY`, `PRDATA` and `PSLVERR` are driven for exactly one cycle, completing the ACCESS phase. Returns to IDLE unconditionally.

Mapping to the bus phases:

APB bus phase	Slave state	PREADY
IDLE (`PSEL=0`)	IDLE	0
SETUP (`PSEL=1`, `PENABLE=0`)	IDLE	0
ACCESS (`PSEL=1`, `PENABLE=1`), wait states	BUSY	0
ACCESS, final cycle	WAIT	1

6.4.3 State Transitions

```
IDLE → BUSY: s_apb_PSEL && s_apb_PENABLE && !r_penable_prev &&
r_cmd_ready
BUSY → WAIT: r_rsp_valid (response available)
WAIT → IDLE: Automatic (1 cycle completion)
```

The `!r_penable_prev` term is a **rising-edge detect on PENABLE**. The command is captured on the SETUP-to-ACCESS transition only, so a multi-cycle ACCESS phase (the normal case, since this slave always inserts wait states) cannot re-issue the same command.

6.4.4 Orphan-Response Guard

APB is strictly one-outstanding: a command is issued on the IDLE-to-BUSY edge and its response is consumed in BUSY. A response sitting in the response skid buffer while the FSM is in IDLE therefore cannot belong to any command. In IDLE the FSM pops and discards it (asserting `r_rsp_ready` without accepting the data) and emits a simulation-only `$display` warning.

This matters because BUSY pairs commands with responses **by position, not by tag**. Without the guard, one stale entry permanently offsets the response stream: every subsequent read returns the previous read's data. Two ways this can happen in practice:

- A backend that emits a duplicate response for a single command.
- An `apb4_slave_cdc` whose two clock domains were reset independently. The current CDC uses gray-pointer FIFOs specifically so it cannot fabricate a transfer this way — see apb4_slave_cdc.md.

6.4.5 Command/Response Flow

```
APB Master → APB Slave Interface → Command Buffer → Command Interface
→ Backend
                    ↓                          ↓                ↓
APB Master ← APB Slave Interface ← Response Buffer ← Response
Interface ← Backend
```

6.4.6 Buffering Architecture

The module uses two GAXI skid buffers for decoupling:

- **Command Buffer**: Stores incoming APB transactions
- **Response Buffer**: Stores backend responses for APB return

6.4.7 Key Features

- **APB4 Compliance**: Full protocol support including PSTRB and PPROT
- **Buffered Operation**: Command and response skid buffers prevent blocking
- **Flow Control**: Proper ready/valid handshaking on all interfaces
- **Error Handling**: PSLVERR propagation from backend to APB
- **Orphan-Response Guard**: Discards responses that arrive with no command outstanding

6.4.8 APB Transaction Timing

State	APB Signals	Internal Operation
IDLE	PREADY=0	Wait for rising edge of PENABLE; drop

State	APB Signals	Internal Operation
		orphan responses
BUSY	PREADY=0	Command issued, wait for backend response
WAIT	PREADY=1, PRDATA/PSLVERR valid	Complete transaction

6.5 Timing Characteristics

6.5.1 Transaction Latency

This slave is **not** a zero-wait-state slave. `PREADY` is registered and is asserted no earlier than FOUR `pclk` cycles after the edge on which `PENABLE` is first sampled high — one cycle IDLE-to-BUSY, one through each skid buffer's registered `rd_valid` (command in, response out), one BUSY-to-WAIT. The ACCESS phase therefore always contains at least four wait states, and a complete APB transfer takes at least six `pclk` cycles end to end against an ideal combinational backend. Backend response latency adds to this directly.

Characteristic	Value	Description
PENABLE rise to PREADY	≥4 clock cycles	Registered FSM path AND both skid buffers register rd_valid, best case
Minimum ACCESS wait states	4	FSM (2) plus one registered rd_valid in each skid buffer
Minimum full transfer	≥6 clock cycles	SETUP + ACCESS with 4 wait states, ideal combinational backend
Response Processing	1+ clock cycles	Backend processing time, additive

6.5.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	200-400 MHz	Technology dependent, not characterized in this repository
Buffer Depth	2 entries	With default DEPTH=2
Outstanding Transactions	1	APB is non-pipelined; buffers absorb backend latency, they do not add concurrency

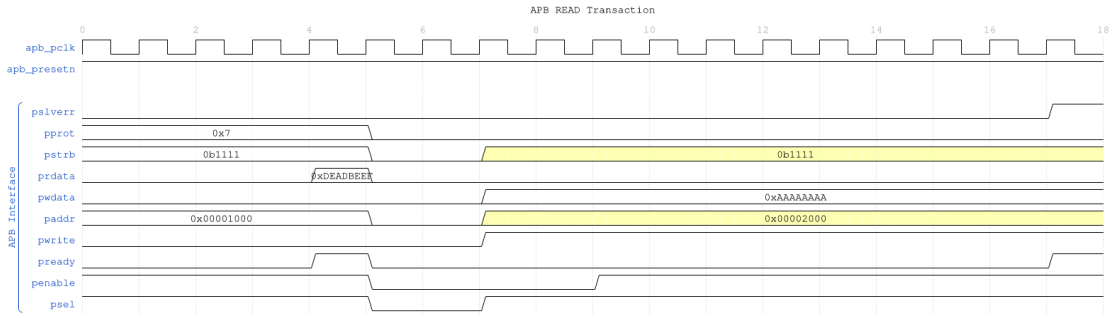
On throughput: APB is a non-pipelined protocol. The skid buffers decouple the APB FSM from backend latency; they do not allow a second APB transfer to start before the first completes. Peak throughput is therefore `DATA_WIDTH / (minimum transfer cycles × pclk period)`, not one transfer per cycle.

6.6 Waveforms

The following timing diagrams show the comprehensive APB slave behavior across 7 scenarios.

Known diagram defect: the WaveDrom generator stamps a fixed header string, `APB READ Transaction`, onto every scenario image regardless of direction. The signal traces themselves are correct — read `PWRITE` in the trace, not the header, to determine transaction direction. This affects the images under `docs/markdown/assets/WAVES/apb4_slave/` and `.../apb4_master/`.

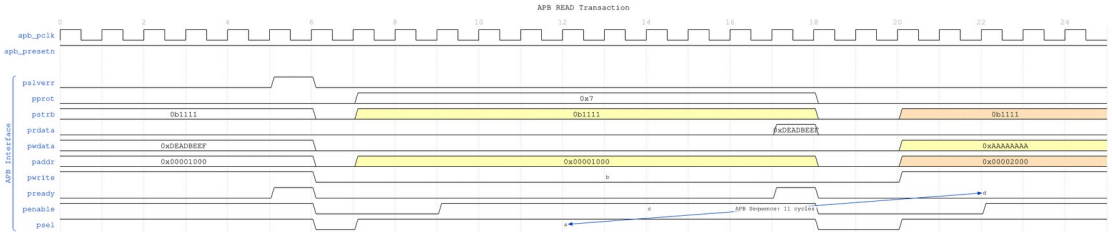
6.6.1 Scenario 1: Basic Write Transaction



APB Write

WaveJSON: apb_write_sequence_001.json

6.6.2 Scenario 2: Basic Read Transaction



APB Read

WaveJSON: apb_read_sequence_001.json

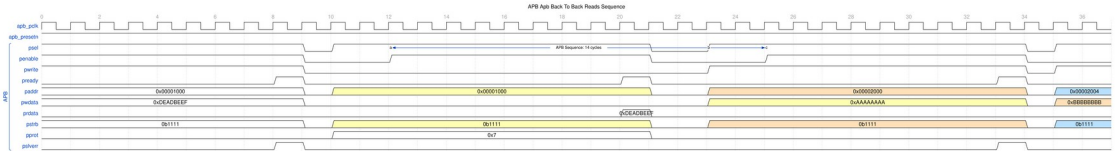
6.6.3 Scenario 3: Back-to-Back Writes



B2B Writes

WaveJSON: apb_back_to_back_writes_001.json

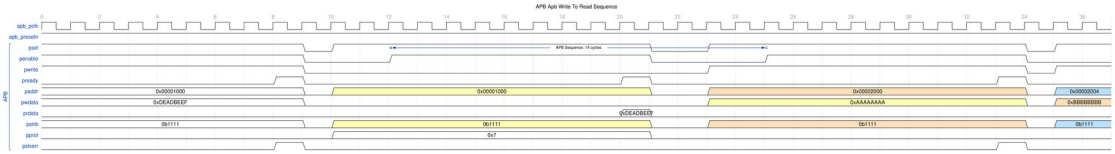
6.6.4 Scenario 4: Back-to-Back Reads



B2B Reads

WaveJSON: apb_back_to_back_reads_001.json

6.6.5 Scenario 5: Write-to-Read Transition



Write-to-Read

WaveJSON: apb_write_to_read_001.json

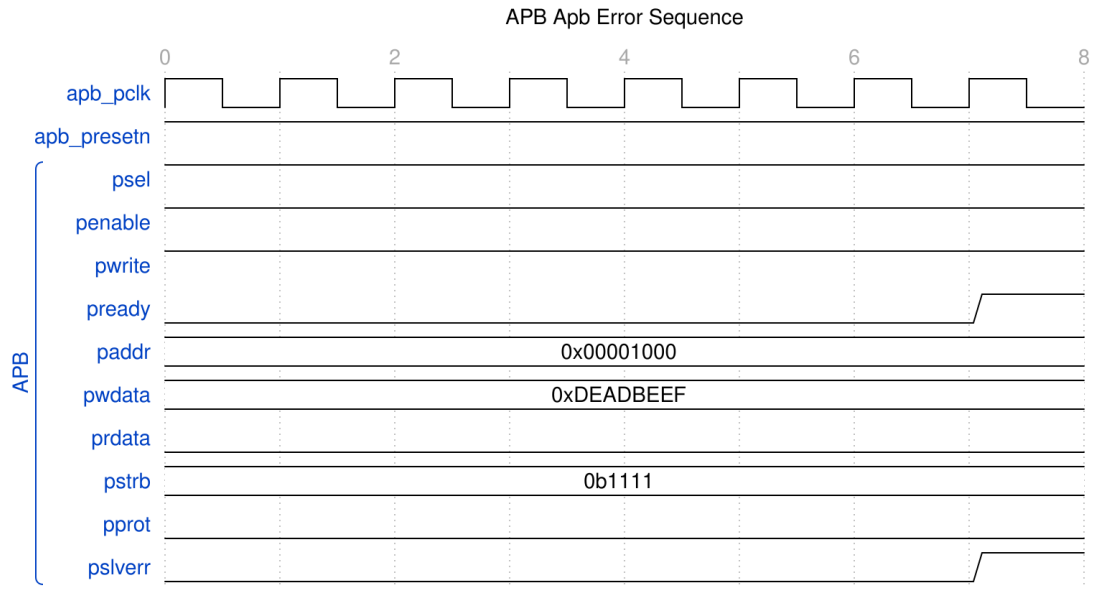
6.6.6 Scenario 6: Read-to-Write Transition



Read-to-Write

WaveJSON: apb_read_to_write_001.json

6.6.7 Scenario 7: Error Response



Error

WaveJSON: apb_error_001.json

6.7 Usage Examples

Every parameter and port below is read from the module declaration.

```
apb4_slave #(
    .ADDR_WIDTH             (32),
    .DATA_WIDTH             (32),
    .PROT_WIDTH             (3),
    .DEPTH                  (2)
) u_apb4_slave (
    .pclk                   (pclk),
    .presetn                (presetn),
    .s_apb_PSEL             (s_apb_PSEL),
    .s_apb_PENABLE          (s_apb_PENABLE),
    .s_apb_PREADY           (s_apb_PREADY),
    .s_apb_PADDR            (s_apb_PADDR),
    .s_apb_PWRITE           (s_apb_PWRITE),
    .s_apb_PWDATA           (s_apb_PWDATA),
    .s_apb_PSTRB            (s_apb_PSTRB),
    .s_apb_PPROT            (s_apb_PPROT),
    .s_apb_PRDATA           (s_apb_PRDATA),
    .s_apb_PSLVERR          (s_apb_PSLVERR),
```

```
    .cmd_valid              (cmd_valid),
    .cmd_ready              (cmd_ready),
    .cmd_pwrite             (cmd_pwrite),
    .cmd_paddr              (cmd_paddr),
    .cmd_pwdata             (cmd_pwdata),
    .cmd_pstrb              (cmd_pstrb),
    .cmd_pprot              (cmd_pprot),
    .rsp_valid              (rsp_valid),
    .rsp_ready              (rsp_ready),
    .rsp_prdata             (rsp_prdata),
    .rsp_pslverr            (rsp_pslverr)
);
```

6.8 Design Notes

6.8.1 Buffer Depth Selection

`DEPTH` is an entry count and must be one of 2..8 inclusive. Because APB allows only one outstanding transfer, extra depth does **not** buy concurrency — it only absorbs jitter in a backend that returns responses unevenly. `DEPTH=2` is correct for almost every backend.

Backend Type	Recommended DEPTH	Rationale
Register Block	2	Single-cycle response; deeper buffers are pure area
SRAM Controller	2	Latency is absorbed by wait states, not by buffering
External Memory	4	Variable response timing; small margin against stalls
Shared/arbitrated backend	4	Response return can be bursty under contention

6.8.2 Clock Domain Optimization

For different clock domains, use `apb4_slave_cdc`:

```
apb4_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32)
) u_cdc_slave (
    // APB clock domain
    .pclk(apb_clk),
    .presetn(apb_resetn),
    .s_apb_PSEL(apb_psel), /* ... remaining s_apb_* ports ... */

    // Backend clock domain
    .aclk(backend_clk),
    .aresetn(backend_resetn),
    .cmd_*(backend_cmd_*),
    .rsp_*(backend_rsp_*)
);
```

The CDC clock/reset ports are `pclk`/`presetn` (APB side) and `aclk`/`aresetn` (backend side) — there is no `s_`/`m_` prefix on them.

6.8.3 Synthesis Considerations

Area: - Reduce DEPTH for area-constrained designs - Share skid buffers across multiple slaves when possible - Use synchronous reset for smaller area

Timing: - Register all command/response outputs - Use appropriate buffer depths to meet timing - Consider pipeline stages for high-frequency operation

Power: - Use clock-gated variant (`apb4_slave_cg`) when available - Implement conditional clock enables for inactive slaves - Size buffers appropriately to minimize switching activity

6.9 Related Modules

- **apb4_slave_cg**: Clock-gated version for power optimization
- **apb4_slave_cdc**: Clock domain crossing variant
- **apb4_master**: Complementary APB master implementation
- **apb5_slave**: APB5 equivalent with `PWAKEUP`, `P*USER` and optional parity
- **gaxi_skid_buffer**: Underlying buffering infrastructure
- **apbx_xbar_1to4** / **apbx_xbar_2to4**: Generated APB crossbars for multi-slave systems

6.10 Testing

6.10.1 Protocol Compliance

- Verify APB setup and access phase timing

- Check PREADY assertion with valid PRDATA/PSLVERR
- Validate proper state machine operation

6.10.2 Buffer Verification

- Test buffer overflow/underflow conditions
- Verify command/response packet integrity
- Check flow control under various load conditions

6.10.3 Backend Integration

- Test various backend response latencies
- Verify error propagation and handling
- Check concurrent transaction handling

6.11 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c — ch. 3 “Transfers”: the Completer side of §3.1 write and §3.3 read transfers (extending ACCESS with `PREADY` low), §3.2 `PSTRB`, §3.4 error response (`PSLVERR`), §3.5 `PPROT`; ch. 4 “Operating States”. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, section “ARM AMBA interconnect protocols suite” (APB bus protocol overview) — textbook treatment (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (valid-ready protocol and valid-ready slices) — textbook treatment of the skid buffer (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

6.12 Navigation

- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

7 apb4_slave_cdc

Module: `apb4_slave_cdc.sv` **Location:** `rtl/amba/apb4/` **Status:** Production Ready

7.1 Overview

The APB Slave CDC (Clock Domain Crossing) module is a complete APB slave interface with the crossing built in: full APB on the `pclk` side, an AXI/GAXI-style backend on the `aclk` side. It's what lets you integrate APB peripherals running at different clock frequencies without metastability becoming your problem.

7.1.1 Key Features

- **Safe CDC:** Gray-pointer asynchronous FIFOs, one per direction
- **Dual Clock Domains:** APB (pclk) and AXI (aclk) operate independently
- Caveat: **reset both domains together** — a one-sided reset is NOT safe; see Reset Behavior
- **Full APB4 Support:** Complete AMBA 4 APB protocol compliance
- **Command/Response Interface:** Clean GAXI-style backend interface
- **Buffered Operation:** Integrated skid buffers for elastic storage

Protocol scope: APB4 only. For APB5 signalling use `apb5_slave_cdc` from `rtl/amba/apb5/` — see the APB5 book.

7.2 Parameters

Parameter	Type	Default	Description
`ADDR_WIDTH`	int	32	APB address bus width
`DATA_WIDTH`	int	32	APB data bus width
`STRB_WIDTH`	int	DATA_WIDTH/8	Write strobe width (calculated)
`PROT_WIDTH`	int	3	APB protection signal width
`DEPTH`	int	2	Skid-buffer depth **in entries** inside the wrapped `apb4_slave`; also the floor for the CDC

Parameter	Type	Default	Description
			FIFO depth
USE_JOHNSON	int	0 (Gray)	CDC-FIFO pointer encoding: 0 Gray (power-of-2 derived depth only), 1 Johnson (any FIFO depth, max(DEPTH,4)-bit pointers; module-level DEPTH stays limited to 2..8 inclusive by the skid buffers). Gray by default — Johnson is opt-in because its pointers cost DEPTH bits in both domains and every synchronizer stage.
USE_2_PHASE_CDC	bit	1	**Deprecated and ignored.** Has no effect on the generated logic

7.3 Ports

```
module apb4_slave_cdc #(
    parameter int ADDR_WIDTH  = 32,
    parameter int DATA_WIDTH  = 32,
    parameter int STRB_WIDTH  = DATA_WIDTH / 8,
    parameter int PROT_WIDTH  = 3,
    parameter int DEPTH       = 2,
    // Async-FIFO pointer encoding: 0 = Gray (power-of-2 derived depth
only),
    // 1 = Johnson (any depth, DEPTH-bit pointers). Forwarded to both
CDC FIFOs.
    parameter int USE_JOHNSON = 0,
    // DEPRECATED / NO EFFECT -- retained only so existing
instantiations
    // still elaborate. See "CDC Implementation" below.
    parameter bit USE_2_PHASE_CDC = 1'b1,
    // Derived width aliases the port list references -- not
independent knobs
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
```

```
    parameter int SW  = STRB_WIDTH,
    parameter int PW  = PROT_WIDTH
) (
    // Clock and Reset
    input  logic              aclk,
    input  logic              aresetn,
    input  logic              pclk,
    input  logic              presetn,

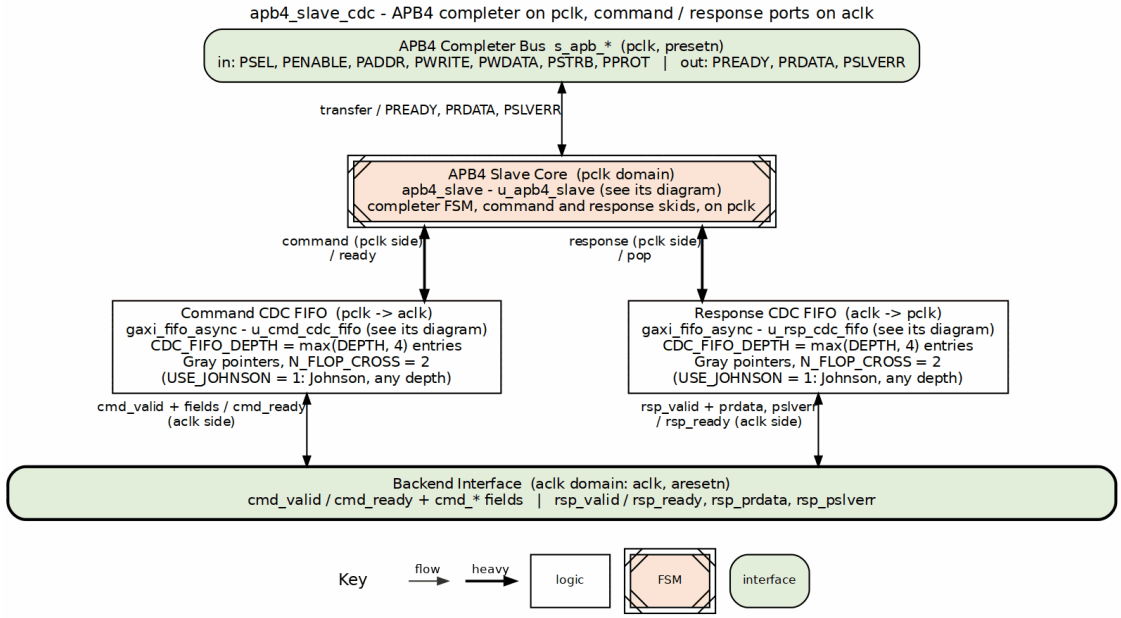
    // APB Slave Interface (pclk domain)
    input  logic              s_apb_PSEL,
    input  logic              s_apb_PENABLE,
    output logic              s_apb_PREADY,
    input  logic [AW-1:0]     s_apb_PADDR,
    input  logic              s_apb_PWRITE,
    input  logic [DW-1:0]     s_apb_PWDATA,
    input  logic [SW-1:0]     s_apb_PSTRB,
    input  logic [PW-1:0]     s_apb_PPROT,
    output logic [DW-1:0]     s_apb_PRDATA,
    output logic              s_apb_PSLVERR,

    // Command Interface (aclk domain)
    output logic              cmd_valid,
    input  logic              cmd_ready,
    output logic              cmd_pwrite,
    output logic [AW-1:0]     cmd_paddr,
    output logic [DW-1:0]     cmd_pwdata,
    output logic [SW-1:0]     cmd_pstrb,
    output logic [PW-1:0]     cmd_pprot,

    // Response Interface (aclk domain)
    input  logic              rsp_valid,
    output logic              rsp_ready,
    input  logic [DW-1:0]     rsp_prdata,
    input  logic              rsp_pslverr
);
```

7.4 Functional Description

7.4.1 Block diagram



apb4_slave_cdc block diagram

The APB side runs on `pclk`: the bus feeds the wrapped `apb4_slave` (one box; see its diagram). Each direction crosses to the `aclk` backend through its own `gaxi_fifo_async`: the command drops down the left lane (pclk to aclk), the response climbs the right lane (aclk to pclk). The crossings themselves – Gray pointers through two-flop synchronizers – are inside the async FIFOs and drawn in the `gaxi_fifo_async` diagram. Both FIFOs are `max(DEPTH, 4)` entries deep.

7.4.2 Clock Domains

APB domain (pclk):

- APB slave interface signals
- Typical frequency: 50-200 MHz
- Used by APB master/interconnect

AXI domain (aclk):

- Command and response interfaces
- Can be faster or slower than pclk
- Used by backend processing logic

7.4.3 CDC Structure

The module is an `apb4_slave` in the `pclk` domain plus two independent asynchronous FIFOs:

FIFO	Direction	Write domain	Read domain	Payload width
`u_cmd_cdc_fifo`	Command	`pclk` / `presetn`	`aclk` / `aresetn`	`CPW = AW + DW + SW + PW + 1`
`u_rsp_cdc_fifo`	Response	`aclk` / `aresetn`	`pclk` / `presetn`	`RPW = DW + 1`

Both are `gaxi_fifo_async` instances with:

- **Pointer encoding:** absolute read/write pointers, Gray by default (`USE_JOHNSON = 0`); pass `USE_JOHNSON = 1` for Johnson
- **Synchronizer depth:** `N_FLOP_CROSS = 2` (two-flop synchronizer per crossed pointer)
- **FIFO depth:** `CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH` — a floor of 4 entries regardless of the `DEPTH` used for the internal skid buffers. **Under the default `USE_JOHNSON = 0` a power of two is required**, because Gray carries a generate-scope elaboration check (`(USE_JOHNSON == 0) && ((DEPTH & (DEPTH-1)) != 0)` -> `$error`). The check sees the DERIVED depth `max(DEPTH, 4)`, while the same `DEPTH` also reaches the wrapped `apb4_slave`'s two `gaxi_skid_buffer` instances UNFLOORED, whose guard accepts 2..8 inclusive (any integer). Under the default Gray encoding the floored depth must be a power of two, so the legal `DEPTH` set is **{2, 3, 4, 8}** (2 and 3 floor to 4); 5, 6 and 7 floor to themselves, fail the power-of-2 check, and elaborate only with `USE_JOHNSON = 1` – Johnson buys exactly those three depths. It pays for that with pointers as wide as the derived FIFO depth `max(DEPTH, 4)` against Gray's `$clog2(max(DEPTH, 4))+1` – at the default `DEPTH = 2` that is 4 bits vs 3, not 2 vs 2 – in both domains and every synchronizer stage; Gray is the default so that cost is never paid by accident.

There is no separate metastability-hardening option — two-flop synchronization is fixed at instantiation.

7.4.4 Reset Behavior

`presetn` and `aresetn` are separate reset domains, but they must be asserted together. **A one-sided reset is not safe. Quiesce the bus first.**

The local reset clears that domain's own pointer, but the crossed copy of the *remote* pointer is a live synchronizer (`glitch_free_n_dff_arn`, N=2) that keeps sampling the non-reset domain the moment reset deasserts — it does not hold at zero. The reset side comes back with its own pointer at zero and the remote pointer at whatever the other side had reached: not an empty FIFO, a mismatched one.

Neither consequence is a clean discard:

- **Consumed commands are re-presented.** Pulsing `aresetn` with an **already-consumed** command in the cmd FIFO rewinds the read pointer behind the write pointer, so the backend sees that command again after reset. It does not time out — it re-executes. An *unread* command is not at risk: its read pointer is already behind the write pointer, so resetting it to 0 rewinds nothing and the entry is delivered exactly once.
- **The response FIFO can fabricate responses.** The same rewind on the response path presents entries the APB side never queued, so the APB master can complete a transfer the backend never answered.

Pointers being absolute positions rather than toggle parity is what makes the *steady-state* crossing reliable; it does not make a one-sided reset safe.

7.4.5 Why Not the Previous 2-Phase Handshake

`USE_2_PHASE_CDC` selected a toggle-based handshake in an earlier revision. It is now deprecated and ignored; the parameter survives only for source-compatibility with existing instantiations.

The failure mode is worth understanding, because it's the kind that ships. A 2-phase handshake encodes each transfer as a **toggle**. Reset the two domains independently and the toggle parity desynchronizes — the link fabricates or drops exactly one transfer. Permanently, because nothing ever re-syncs it. Pair that with the `apb4_slave` FSM, which pairs commands and responses by position rather than by tag, and a single phantom transfer offsets the response stream by one forever: every read returns the previous read's data.

This was observed on the Nexys A7 `ddr2-char` board on 2026-07-19. Reading a single CSR eight times returned the previous register's value about three times before settling, while the non-CDC harness window was stable. The two mitigations now in place are the gray-pointer FIFOs described above and the orphan-response guard in `apb4_slave` — see apb4_slave.md.

7.5 Design Notes

This is a clock-domain crossing; the usual CDC rules bind. Pointers cross Gray-coded through `glitch_free_n_dff_arn`, and the reset must be asserted in both domains – releasing one before the other leaves the pointer comparison meaningless until the second releases.

Depth buys latency tolerance, not bandwidth. A CDC FIFO's throughput is set by the slower domain; a deeper buffer absorbs a longer burst without stalling but does not raise the sustained rate. See `vault/handbook/design/cdc.md`.

7.6 Timing Characteristics

7.6.1 Maximum Clock Ratio

There is no maximum ratio between `pclk` and `aclk`. Gray-pointer FIFOs impose no relationship between the two clocks — either may be arbitrarily faster, slower, or phase-unrelated, and either may be stopped indefinitely. Stopping `aclk` simply stalls the command FIFO's read side; the APB side backpressures via `PREADY` held low and no data is lost.

What the ratio affects is throughput, not correctness. Each transfer pays the usual two-flop synchronizer latency in each direction (roughly 2-3 destination clock edges per crossing), so a very slow `aclk` directly lengthens APB wait states.

7.6.2 Timing Constraints

The crossed signals are the gray-coded pointers and the FIFO memory read path. Standard practice applies:

- Declare `pclk` and `aclk` asynchronous to each other (`set_clock_groups -asynchronous`).
- Do not over-constrain the pointer synchronizer paths; the gray encoding tolerates a one-bit-at-a-time skew by construction.

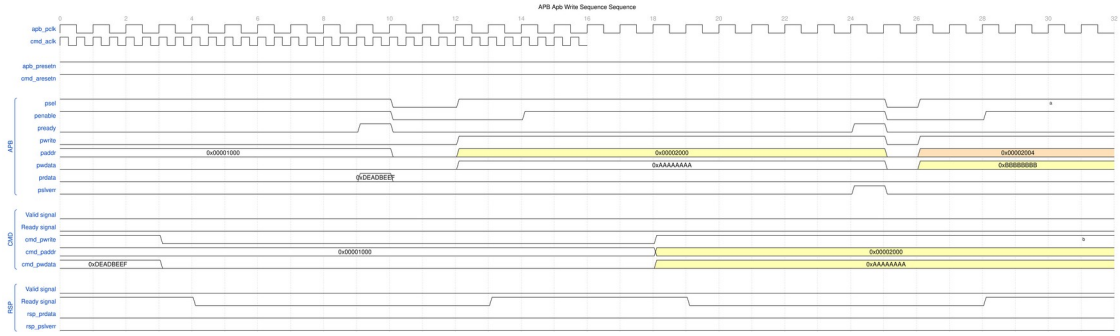
7.7 Waveforms

The following timing diagrams show CDC behavior with **both clock domains visible**:

Clock Configuration: - `apb_pclk`: 100MHz (10ns period) - `cmd_aclk`: 500MHz (2ns period), displayed with `period=0.4` for visual compactness

7.7.1 Scenario 1: Write Transaction with CDC

Shows APB write crossing from pclk domain to aclk domain:



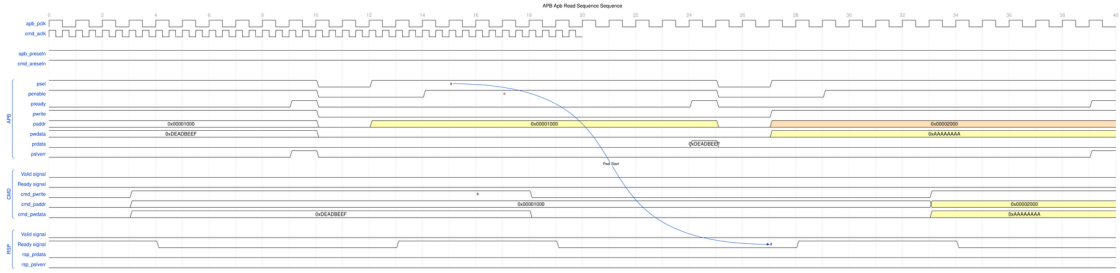
Write CDC

WaveJSON: apb_write_sequence_001.json

Key Observations: - APB transaction in pclk domain - CMD transaction crosses to aclk domain - Note the CDC latency between domains

7.7.2 Scenario 2: Read Transaction with CDC

Shows APB read with response crossing back from aclk to pclk domain:

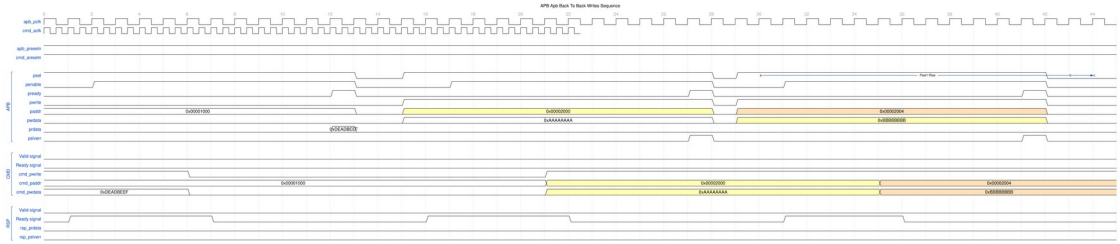


Read CDC

WaveJSON: apb_read_sequence_001.json

Key Observations: - APB read request in pclk domain - CMD crosses to aclk domain - RSP crosses back to pclk domain - Complete round-trip CDC visible

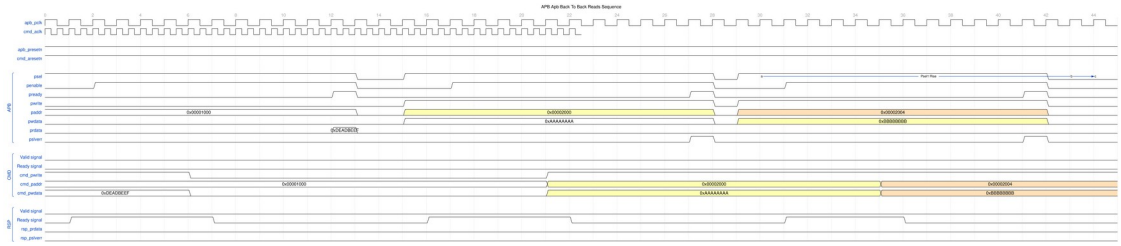
7.7.3 Scenario 3: Back-to-Back Writes with CDC



B2B Writes CDC

WaveJSON: apb_back_to_back_writes_001.json

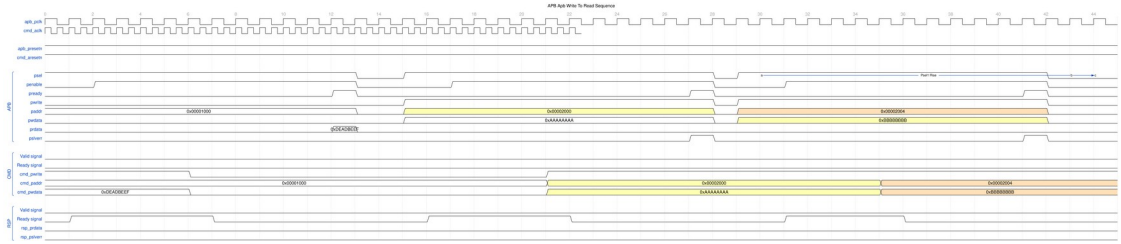
7.7.4 Scenario 4: Back-to-Back Reads with CDC



B2B Reads CDC

WaveJSON: apb_back_to_back_reads_001.json

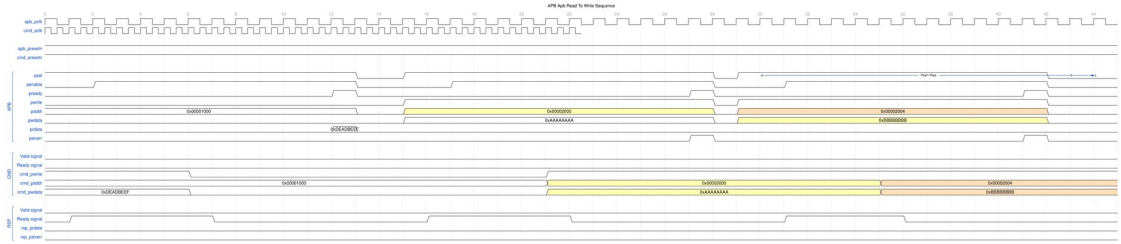
7.7.5 Scenario 5: Write-to-Read Transition with CDC



Write-to-Read CDC

WaveJSON: apb_write_to_read_001.json

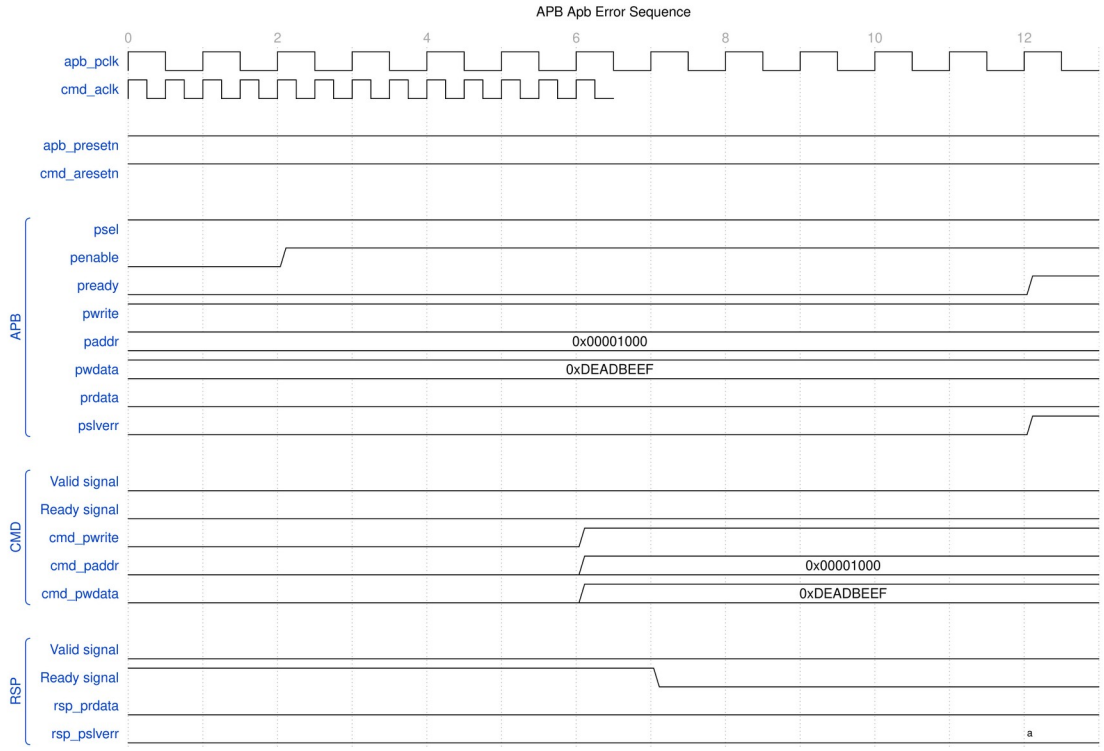
7.7.6 Scenario 6: Read-to-Write Transition with CDC



Read-to-Write CDC

WaveJSON: apb_read_to_write_001.json

7.7.7 Scenario 7: Error Response with CDC



Error CDC

WaveJSON: apb_error_001.json

7.8 Usage Examples

```
apb4_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb_cdc (
    // APB clock domain
    .pclk         (apb_clk),
    .presetn      (apb_resetn),

    // AXI clock domain
    .aclk         (axi_clk),
    .aresetn      (axi_resetn),

    // APB slave interface (pclk domain)
    .s_apb_PSEL     (apb_psel),
```

```
    .s_apb_PENABLE  (apb_penable),
    .s_apb_PREADY   (apb_pready),
    .s_apb_PADDR    (apb_paddr),
    .s_apb_PWRITE   (apb_pwrite),
    .s_apb_PWDATA   (apb_pwdata),
    .s_apb_PSTRB    (apb_pstrb),
    .s_apb_PPROT    (apb_pprot),
    .s_apb_PRDATA   (apb_prdata),
    .s_apb_PSLVERR  (apb_pslverr),

    // Command interface (aclk domain)
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),

    // Response interface (aclk domain)
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr)
);
```

7.9 Related Modules

- **APB Slave:** apb4_slave.md
- **Clock-Gated Variant:** apb4_slave_cdc_cg.md
- **APB5 Equivalent:** apb5_slave_cdc.md
- **CDC FIFO:** `rtl/cdc/gaxi_fifo_async.sv`

7.10 Testing

`val/amba/test_apb4_slave_cdc.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb4_slave_cdc.py -v
```

7.11 References

- **Source:** `rtl/amba/apb4/apb4_slave_cdc.sv`
- **Tests:** `val/amba/test_apb4_slave_cdc.py`
- **WaveDrom Test:** `val/amba/test_apb4_slave_cdc.py::test_apb4_slave_cdc_wavedrom`
- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – ch. 3 “Transfers”: the Completer side of §3.1 write and §3.3 read transfers (extending ACCESS with `PREADY` low), §3.2 `PSTRB`, §3.4 error response (`PSLVERR`), §3.5 `PPROT`; ch. 4 “Operating States”. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 – synchronizer depth versus MTBF for the pointer crossings.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`)

Last Updated: 2026-07-19

7.12 Navigation

- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

8 apb4_slave_cdc_cg

Module: `apb4_slave_cdc_cg.sv` **Base Module:** apb4_slave_cdc **Location:** `rtl/amba/apb4/`
Status: Production Ready

8.1 Overview

This is the **clock-gated variant** of apb4_slave_cdc.

For the clock-gating architecture and the underlying gate cell, see:

→ Clock-Gated Variants Guide and **→ amba_clock_gate_ctrl**

Both live in the Shared book, which is published as a separate PDF. Take the parameter and port names for this module from the tables below, not from the generic guide.

The `apb4_slave_cdc_cg` module adds power optimization to `apb4_slave_cdc` through activity-based clock gating. **Both clock domains are gated independently** — the module instantiates two `amba_clock_gate_ctrl` blocks, one on `pclk` and one on `aclk`, sharing the same configuration inputs.

Structurally it's a sibling of `apb4_slave_cdc`, not a wrapper around it: it re-instantiates the same `apb4_slave` plus the same pair of `gaxi_fifo_async` CDC FIFOs, but drives them from the gated clocks. The CDC behaviour described in apb4_slave_cdc.md — gray pointers, `N_FLOP_CROSS=2`, no maximum clock ratio — applies unchanged. That includes the reset rule: a one-sided reset is NOT safe here either, for the same reason (the crossed pointer copy is a live synchronizer, not a snapshot).

While a domain is gated, that domain's `ready` outputs are forced low, so no handshake can complete against a stopped clock.

- **Same Functionality:** 100% equivalent to base module
- **Dual-Domain Gating:** Separate gate cell per clock domain
- **Runtime Control:** Gating is enabled and tuned by input signals, not parameters
- **Bypassable:** Tie `cfg_cg_enable = 0` for behaviour identical to the base module

---

8.2 Parameters

In addition to all apb4_slave_cdc parameters:

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown counter, shared by both domains

USE_2_PHASE_CDC is inherited from the base module and is likewise **deprecated and ignored**. There is no ENABLE_CLOCK_GATING parameter and no per-domain CG_GATE_* parameters.

8.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
STRB_WIDTH	DATA_WIDTH / 8
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
CPW	AW + DW + SW + PW + 1
RPW	DW + 1

8.3 Ports

In addition to all apb4_slave_cdc ports:

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Global clock-gate enable for both domains. 0 = never gate
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating
cg_gating	1	Output	Asserted while the pclk domain is gated
cg_idle	1	Output	Asserted one cycle after the pclk-domain activity terms go low. **Not an occupancy flag** — see

Port	Width	Direction	Description
			below
`cg_gating`	1	Output	Asserted while the `aclk` domain is gated
`cg_idle`	1	Output	Asserted one cycle after the `aclk`-domain activity terms go low. **Not an occupancy flag** — see below

`*_cg_idle` does not know whether the FIFOs are empty. `amba_clock_gate_ctrl` has no occupancy input at all — its whole idle logic is `r_wakeup <= user_valid || axi_valid;` and `assign idle = ~r_wakeup`. So idle asserts one cycle after the activity terms drop, whatever the CDC FIFOs hold. On this wrapper the activity terms mostly hide occupancy anyway: APB holds `s_apb_PSEL` until `PREADY`, and this slave asserts `PREADY` only when the response returns, so a stalled backend keeps `pclk_user_valid` high and `cg_idle` low for the whole stall; an unconsumed response likewise holds `w_rsp_valid`/`rsp_valid` high. Do not read that as "idle implies empty" — it does not, here or anywhere: idle is "nothing has been handed to me recently", the input to the gating countdown.

Do not use it as a safe-to-reset or safe-to-power-down qualifier; that requires an emptiness check this signal does not perform.

8.3.1 Clock-gating status outputs

This wrapper spans TWO clock domains and gates each independently, so it carries three pairs rather than one. They are not interchangeable.

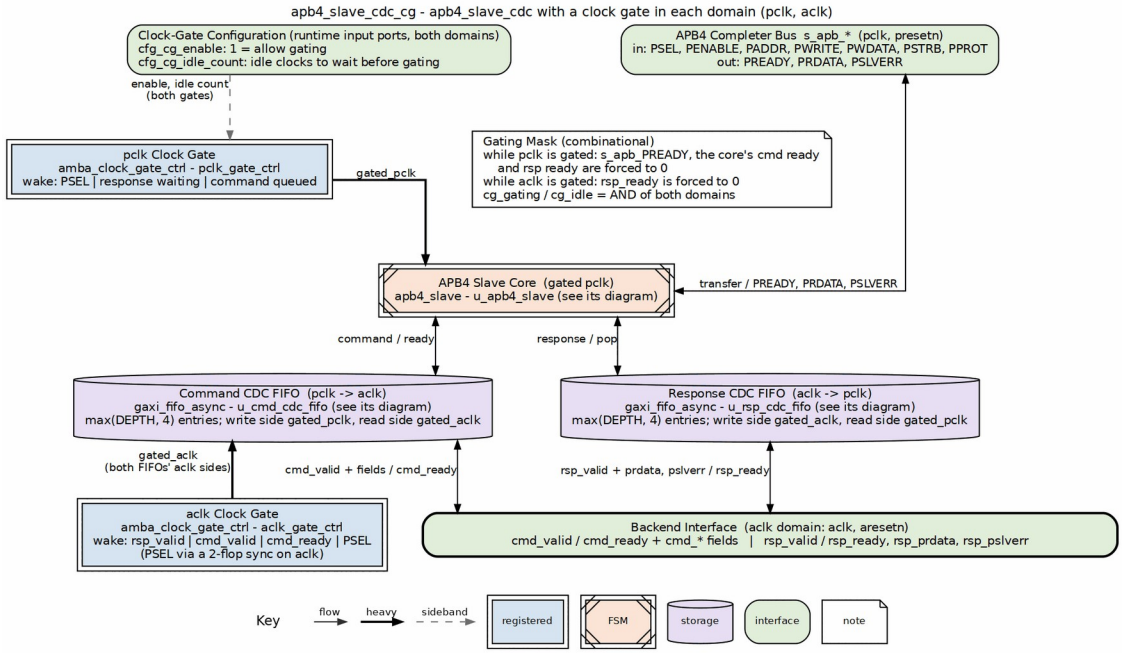
Port	Dir	Width	Description
`pclk_cg_gating`	Out	1	APB-domain clock gated. Masks `s_apb_PREADY` and the command-side ready.
`pclk_cg_idle`	Out	1	APB-domain

Port	Dir	Width	Description
			activity terms quiet.
`aclk_cg_gating`	Out	1	AXI-domain clock gated. Masks the response-side ready.
`aclk_cg_idle`	Out	1	AXI-domain activity terms quiet.
`cg_gating`	Out	1	Whole-module aggregate: `pclk_cg_gating` && `aclk_cg_gating`. High only when BOTH domains are gated.
`cg_idle`	Out	1	Whole-module aggregate: `pclk_cg_idle` && `aclk_cg_idle`.

Use the per-domain signals to reason about one side's readys; use `cg_gating` only as the "is this block asleep" summary. Treating `cg_gating` as equivalent to `pclk_cg_gating` is wrong: with one domain busy the aggregate stays low while that domain's readys are still masked.

8.4 Functional Description

8.4.1 Block diagram



apb4_slave_cdc_cg block diagram

`apb4_slave_cdc` with a clock gate in each domain. The pclk gate's gated clock (heavy) runs the `apb4_slave` core and the pclk sides of both CDC FIFOs; the aclk gate's gated clock runs their aclk sides. Each gate wakes on its own domain's activity (the aclk gate sees PSEL through a two-flop synchronizer). While a domain is gated, the ready signals it drives are forced to 0 (the Gating Mask legend), and `cg_gating` / `cg_idle` are the AND of both domains.

This wrapper is the base module plus one `amba_clock_gate_ctrl` instance. The datapath is untouched – every channel signal is forwarded verbatim – so functional behaviour is identical to the base module and the wrapper adds no latency of its own.

What it adds is a gated clock. `amba_clock_gate_ctrl` watches two activity terms, `user_valid` (this side's valids plus the base module's `busy`) and `axi_valid` (the far side's valids), registers their OR into `r_wakeup`, and stops the inner module's clock once both have been quiet for `cfg_cg_idle_count` cycles. The clock restarts on the next activity, one cycle later.

While the clock is stopped the wrapper masks its outward-facing READY signals with `! cg_gating`, so a peer sees no acceptance until the clock runs again and no handshake is lost across the wake boundary.

`cfg_cg_enable` arms this behaviour; with it low the clock free-runs and the module is indistinguishable from its base.

8.5 Timing Characteristics

This module is **sequential**: it contains 1 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

8.6 Usage Examples

```
apb4_slave_cdc_cg #(
    // Base module parameters (see apb4_slave_cdc.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .DEPTH               (2),
    .USE_JOHNSON         (0),   // 0 = Gray: DEPTH in {2,3,4,8}
(floored max(DEPTH,4) must be pow2); 1 = Johnson: any DEPTH 2..8

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),
    .aclk              (core_clk),
    .aresetn           (core_resetn),

    // Clock gating control (applies to both domains)
    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd8),

    // Per-domain gating status
    .cg_gating    (pclk_gated),
    .cg_idle      (pclk_idle),
    .cg_gating    (aclk_gated),
    .cg_idle      (aclk_idle),

    // ... all other ports same as apb4_slave_cdc
);
```

Simulation note: set `cfg_cg_enable = 0` when debugging CDC waveforms. Two independently gated clocks make cross-domain timing very hard to read.

8.7 Design Notes

A peer's READY must never enter the activity term. A consumer that parks its response-ready high while idle is behaving correctly; folding that signal into `user_valid` pins this block permanently awake and defeats gating entirely – silently, because function is unaffected. Ten wrappers in this repository shipped that way and nothing failed until someone measured. `val/amba/test_cg_peer_ready.py` parks the peer READY high, holds every VALID low, and requires `cg_gating`. Canonical rule: `vault/handbook/design/clock-gating-activity-terms.md`.

`cfg_cg_enable` is not a kill switch. It arms gating and reaches `amba_clock_gate_ctrl` only; the datapath and any monitor enables are forwarded untouched. With it low the clock free-runs and this module behaves exactly like its base.

Gating latency. The clock stops `cfg_cg_idle_count` + 2 cycles after the last bus activity – the idle counter, plus one for the `r_wakeup` flop. Size the idle count against your traffic's inter-burst gap: too small and the block wakes constantly, too large and it never gates.

Cost. Five flops: `r_wakeup` plus `r_idle_counter` at `IDLE_CNTR_WIDTH`, scaling as 1 + `CG_IDLE_COUNT_WIDTH`. The ICG itself is a latch or BUFGCE, not fabric flops.

8.8 Related Modules

- **Base Module Functionality:** apb4_slave_cdc.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb4_slave_cg.md (APB interface)

8.9 Testing

`val/amba/test_apb4_slave_cdc_cg.py` exercises this module. It collects 2 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_apb4_slave_cdc_cg.py -v
```

8.10 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c — ch. 3 “Transfers”: the Completer side of §3.1 write and §3.3 read transfers (extending ACCESS with `PREADY` low), §3.2 `PSTRB`, §3.4 error response (`PSLVERR`), §3.5 `PPROT`; ch. 4 “Operating States”. Citation only.
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 — the Gray-pointer async FIFO (`gaxi_fifo_async`) that carries the command and response across the two clocks.
- Ginosar, R. “Metastability and Synchronizers: A Tutorial.” *IEEE Design & Test of Computers*, 28(5), Sept./Oct. 2011. DOI: 10.1109/MDT.2011.113 — synchronizer depth versus MTBF for the pointer crossings.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 — the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 — the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Deschamps, J.-P., Sutter, G.D., Cantó, E. *Guide to FPGA Implementation of Arithmetic Functions*, Springer (LNEE 149), 2012, ch. 5 “Electronic Aspects of Digital Design”, §5.3.5.3 “Asynchronous FIFO” — textbook treatment (secondary source).
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” — textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/cdc/references/AsyncFifoGrayPointers__Cummings2002.pdf`, `rtl/cdc/references/MetastabilitySynchronizers__Ginosar2011.pdf`, `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

8.11 Navigation

- ← Back to APB Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

9 apb4_slave_cg

Module: `apb4_slave_cg.sv` **Base Module:** apb4_slave **Location:** `rtl/amba/apb4/` **Status:** Production Ready

---

9.1 Overview

The `apb4_slave_cg` module is a clock-gated variant of apb4_slave that adds comprehensive power optimization capabilities through activity-based clock gating.

9.1.1 Key Differences from Base Module

- **Activity-Based Clock Gating:** Gates `pclk` when the interface has been idle
- **Runtime Configuration:** Enable and idle threshold are input signals, not parameters
- **Gating Status Output:** `cg_gating` reports when the clock is gated
- **Zero Functional Impact:** Maintains 100% functional equivalence with base module

The module is a thin wrapper: one `amba_clock_gate_ctrl` instance produces `gated_pclk`, which feeds an otherwise unmodified `apb4_slave`. There is a **single** gate cell — there are no separate data-path and control-path gating domains.

All other functionality is identical to the base module. See apb4_slave.md for complete functional specification.

9.1.2 When to Use the Clock-Gated Variant

Use `apb4_slave_cg` when: - Power consumption is a critical concern - Design has periods of inactivity (burst traffic patterns) - FPGA/ASIC has integrated clock gating support - Meeting power budgets for battery-operated systems

Use the base module (`apb4_slave`) when: - Maximum performance with no power constraints - Continuous high-activity traffic - Simpler design with fewer configuration parameters - Minimizing gate count is priority

---

9.2 Parameters

In addition to all parameters from apb4_slave, this module adds:

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown counter; bounds the maximum programmable idle threshold

That is the only additional parameter. Gating is not parameterized on or off, and there are no per-domain gating parameters — both are controlled at runtime.

9.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
STRB_WIDTH	DATA_WIDTH / 8
DW	DATA_WIDTH
AW	ADDR_WIDTH
SW	STRB_WIDTH
PW	PROT_WIDTH
ICW	CG_IDLE_COUNT_WIDTH
CPW	AW + DW + SW + PW + 1
RPW	DW + 1

9.3 Ports

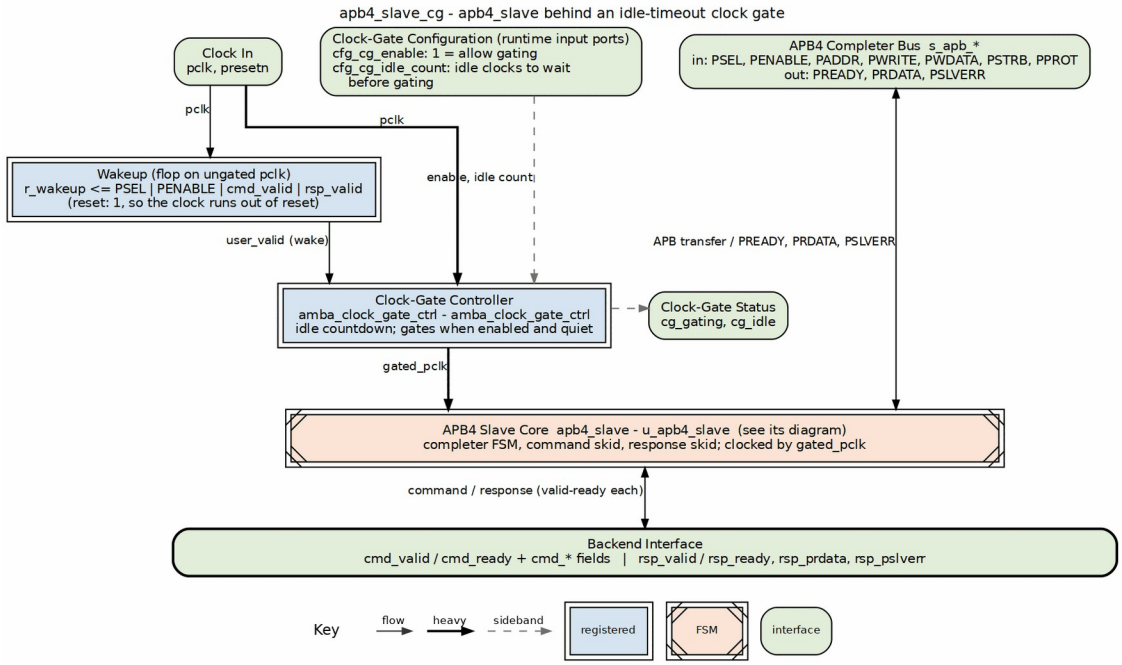
In addition to all ports from apb4_slave:

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Global clock-gate enable. 0 = never gate (identical to base module)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles to count down before gating the clock
cg_gating	1	Output	Asserted while the internal clock is gated
cg_idle	Out	1	Activity terms quiet

Port	Width	Direction	Description
			(registered ~wakeup).

9.4 Functional Description

9.4.1 Block diagram



apb4_slave_cg block diagram

The wrapped `apb4_slave` is one box here; its own diagram shows the inside. The variant adds the clock path (heavy): `pclk` feeds `amba_clock_gate_ctrl`, whose gated clock runs the core. A wakeup flop on the ungated clock watches PSEL, PENABLE, `cmd_valid` and `rsp_valid`; after `cfg_cg_idle_count` quiet clocks (with `cfg_cg_enable` set) the clock stops. `cg_gating` and `cg_idle` report the state.

9.4.2 Single Gating Domain

There is one `amba_clock_gate_ctrl` instance producing one gated clock, which feeds the entire wrapped `apb4_slave`. There is no separate data-path/control-path split.

9.4.3 Wake-Up Condition

The wrapper holds the clock ungated while any of the following is true, sampled one cycle earlier into an internal `r_wakeup` register:

```
s_apb_PSEL || s_apb_PENABLE || cmd_valid || rsp_valid
```

That is: an APB transfer selected or in its ACCESS phase, a command still presented to the backend, or a response still pending. Gating engages `cfg_cg_idle_count + 1` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after all four terms go low, because APB adds two register stages ahead of the ICG enable.

9.4.4 Gating State Machine

The gate cell follows the standard three-state sequence:

```
ACTIVE  IDLE_COUNT  GATED



        (Activity Detected)

States:
- ACTIVE:     Clock enabled, monitoring activity
- IDLE_COUNT: Counting cfg_cg_idle_count cycles before gating
- GATED:      Clock disabled, waiting for activity
```

9.4.5 Wake-Up Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. This wrapper registers activity into its own `r_wakeup` flop and then hands it to `amba_clock_gate_ctrl`, which registers it again, so APB is a **two-stage** family. The first gated-clock rising edge available to the wrapped `apb4_slave` therefore arrives **3 cycles** after activity asserts.

`clock_gate_ctrl` itself adds no flop: `w_gate_enable` is combinational from `wakeup` into the ICG enable, so the third cycle is the released clock edge rather than a further register stage.

`cfg_cg_idle_count` controls how long the clock stays running *after* traffic stops, not how long wake-up takes — a larger value means fewer gate/ungate transitions, not slower wake-up.

9.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

9.6 Usage Examples

9.6.1 Aggressive Gating (Bursty Traffic)

```
apb4_slave_cg #(
    // Base parameters (see apb4_slave.md)
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .DEPTH               (2),
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd4),   // gate quickly after 4 idle cycles
    .cg_gating  (cg_active),
    // ... connect signals same as base module
);
```

9.6.2 Conservative Gating

```
apb4_slave_cg #(
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    .cfg_cg_enable     (1'b1),
    .cfg_cg_idle_count (4'd15),  // wait longer; fewer gate/ungate
events
    .cg_gating  (cg_active),
    // ... connect signals same as base module
);
```

Note that `cfg_cg_idle_count` is bounded by `CG_IDLE_COUNT_WIDTH`. With the default width of 4, the largest usable threshold is 15 cycles; widen the parameter if a longer idle window is required.

9.6.3 Clock Gating Disabled (Functional Verification)

```
apb4_slave_cg #(
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32)
) u_cg (
    .pclk              (apb_clk),
    .presetn           (apb_resetn),

    .cfg_cg_enable     (1'b0),   // never gate
    .cfg_cg_idle_count (4'd0),
    // ... connect signals same as base module
);
```

Note: With `cfg_cg_enable = 0`, this module is functionally identical to the base module.

9.7 Design Notes

9.7.1 Power Savings

Clock gating removes the switching power of the wrapped `apb4_slave` during the gated window. The achievable saving is therefore bounded by the fraction of time the interface is idle, and by whether the target technology maps the enable onto a real clock-gating cell (an ASIC ICG, a Xilinx `BUFGCE`, an `ALTCLKCTRL`) rather than a data-path enable.

No power measurements have been taken for this module in this repository. Any specific percentage would be a guess. If a power number is needed, run vendor power analysis on the target device with a representative traffic switching activity file (SAIF/VCD) and compare against `apb4_slave`.

9.7.2 Gating Status Signal

Signal	Width	Description
`cg_gating`	1	Asserted while the internal clock is gated. Integrate over a window to measure the gated duty cycle

9.7.3 Synthesis Considerations

The gated clock is produced inside `amba_clock_gate_ctrl`. Whether it becomes a true gated clock or a fan-out of clock enables is a synthesis decision:

Xilinx: - Vivado typically converts the enable into `BUFGCE` or into per-flop clock enables - Confirm which happened in the post-synthesis netlist before claiming power savings - Verify with post-implementation power analysis

Quartus: - Maps to `ALTCLKCTRL`; may need explicit vendor primitive instantiation - Check power reports for gating effectiveness

Lattice: - Basic clock gating supported - May require manual instantiation of clock enables - Verify functionality in timing simulation

ASIC: - Work with foundry to select appropriate clock gating cells - Integrated Clock Gating (ICG) cells provide best results - Consider hold-time implications of clock gating - Verify power intent with UPF (Unified Power Format)

9.8 Related Modules

- **apb4_slave** - Base module (non-clock-gated)
- **Power Optimization Guide:** `docs/POWER_OPTIMIZATION_GUIDE.md`
- **Clock Gating Best Practices:** `docs/CLOCK_GATING_GUIDE.md`
- **AMBA Subsystem Overview:** `docs/markdown/rtl-amba/overview.md`

9.9 Testing

9.9.1 Clock Gating in Simulation

Recommendation: Disable clock gating during functional verification:

```
// Testbench instantiation
apb4_slave_cg dut (
    .cfg_cg_enable (1'b0),   // Disable gating for functional debug
    // ... connections
);
```

Rationale: - Simpler waveforms (no clock gating events) - Easier debug: a gated clock freezes internal state and reads like a hang

9.9.2 Power Analysis Verification

For power-specific verification:

1. **Enable gating** (`cfg_cg_enable = 1`) with a realistic `cfg_cg_idle_count`
2. **Monitor `cg_gating`** to verify the expected gated duty cycle
3. **Vary traffic patterns** to test gating effectiveness
4. **Check wake-up timing** meets system requirements

9.10 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c — ch. 3 “Transfers” and ch. 4 “Operating States”, the protocol of the wrapped APB4 block. Citation only.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 — the fixed-timeout policy `amba_clock_gate_ctrl` implements: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. CAD*, 15(6), June 1996. DOI: 10.1109/43.503933 — the latch-based gated clock (`icg`) at the bottom of the gating chain.
- Roy, S. *Advanced Digital System Design*, Springer, ch. 15 “Low-Power Digital System Design”, §15.5.5 “Logic Level Power Down” — textbook treatment of clock gating (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `rtl/common/references/DynamicPowerManagementTimeout__Benini2000.pdf`, `rtl/common/references/LatchBasedClockGating__Benini1996.pdf`)

9.11 Navigation

- **← Back to Base Module**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

10 apb4_slave_stub

10.1 Overview

The `apb4_slave_stub` is a lightweight APB slave that converts APB protocol signals into packed command/response packets. It exists for testbench use — when you need a simple APB slave without the full functionality of the standard `apb4_slave` module. The packed interface simplifies integration with test responders and verification components.

10.2 Parameters

Parameter	Type	Default	Description
DEPTH	int	4	Skid-buffer depth in **entries** (not a log2 exponent); must be one of 2..8 inclusive
DATA_WIDTH	int	32	APB data bus width
ADDR_WIDTH	int	32	APB address bus width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
CMD_PACKET_WIDTH	int	Calculated	Command packet width
RESP_PACKET_WIDTH	int	Calculated	Response packet width

10.3 Ports

```
module apb4_slave_stub #(
    parameter int DEPTH    = 4,
    parameter int DATA_WIDTH    = 32,
    parameter int ADDR_WIDTH    = 32,
    parameter int STRB_WIDTH    = DATA_WIDTH / 8,
    parameter int CMD_PACKET_WIDTH = ADDR_WIDTH + DATA_WIDTH +
STRB_WIDTH + 4, // addr, data, strb, prot, pwrite
    parameter int RESP_PACKET_WIDTH = DATA_WIDTH + 1, // data, resp
    // Short Parameters
    parameter int DW  = DATA_WIDTH,
    parameter int AW  = ADDR_WIDTH,
    parameter int SW  = STRB_WIDTH,
    parameter int CPW = CMD_PACKET_WIDTH,
    parameter int RPW = RESP_PACKET_WIDTH
) (
```

```
    // Clock and Reset
    input  logic                        pclk,
    input  logic                        presetn,

    // APB Slave Interface
    input  logic                        s_apb_PSEL,
    input  logic                        s_apb_PENABLE,
    input  logic [AW-1:0]               s_apb_PADDR,
    input  logic                        s_apb_PWRITE,
    input  logic [DW-1:0]               s_apb_PWDATA,
    input  logic [SW-1:0]               s_apb_PSTRB,
    input  logic [2:0]                  s_apb_PPROT,
    output logic [DW-1:0]               s_apb_PRDATA,
    output logic                        s_apb_PSLVERR,
    output logic                        s_apb_PREADY,

    // Command Packet Interface (packed)
    output logic                        cmd_valid,
    input  logic                        cmd_ready,
    output logic [CPW-1:0]              cmd_data,

    // Response Packet Interface (packed)
    input  logic                        rsp_valid,
    output logic                        rsp_ready,
    input  logic [RPW-1:0]              rsp_data
);
```

10.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

10.3.2 APB Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	Peripheral select
s_apb_PENABLE	1	Input	Enable signal
s_apb_PADDR	AW	Input	Address bus
s_apb_PWRITE	1	Input	Write/read (1=write,

Port	Width	Direction	Description
			0=read)
s_apb_PWDATA	DW	Input	Write data
s_apb_PSTRB	SW	Input	Write strobe
s_apb_PPROT	3	Input	Protection attributes
s_apb_PRDATA	DW	Output	Read data
s_apb_PSLVERR	1	Output	Slave error
s_apb_PREADY	1	Output	Ready signal

10.3.3 Packed Command Interface

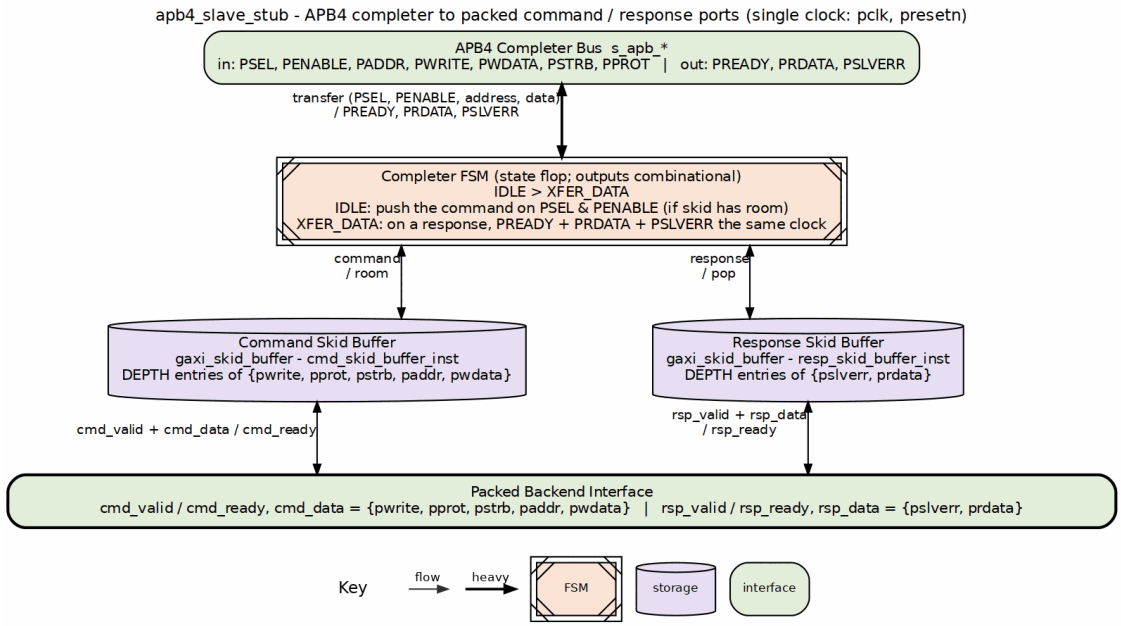
Port	Width	Direction	Description
cmd_valid	1	Output	Command packet valid
cmd_ready	1	Input	Ready for command packet
cmd_data	CPW	Output	Packed command (pwrite, pprot, pstrb, paddr, pwdata)

10.3.4 Packed Response Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response packet valid
rsp_ready	1	Output	Ready for response packet
rsp_data	RPW	Input	Packed response (resp, prdata)

10.4 Functional Description

10.4.1 Block diagram



apb4_slave_stub block diagram

A standalone completer (it does not instantiate `apb4_slave`). The APB bus enters at the top; a two-state FSM (IDLE > XFER_DATA) pushes the command on PSEL and PENABLE into the command skid, which feeds the packed backend interface. The response climbs the right lane through the response skid, and in XFER_DATA the FSM drives PREADY, PRDATA and PSLVERR combinationally in the same clock it pops it.

10.4.2 Packet Interface

The stub uses packed interfaces to simplify testbench integration:

Command Packet Format (MSB to LSB): - `pwrite` (1 bit): Write/read operation - `pprot` (3 bits): Protection attributes - `pstrb` (SW bits): Write strobe - `paddr` (AW bits): Address - `pwdata` (DW bits): Write data — the LSB field (paddr sits ABOVE pwdata, matching the comparison table below)

Response Packet Format (MSB to LSB): - `pslverr` (1 bit): Slave error - `prdata` (DW bits): Read data

10.4.3 Packet Format Is Not Symmetric With `apb4_master_stub`

`apb4_master_stub` carries two extra bits in each direction — `first` and `last`:

	apb4_slave_stub	apb4_master_stub
CMD_PACKET_WIDTH	AW + DW + SW + 4	AW + DW + SW + 3 + 1 + 1 + 1
Command fields	pwrite, pprot, pstrb, paddr, pwdata	last, first, pwrite, pprot, pstrb, paddr, pwdata
RESP_PACKET_WIDTH	DW + 1	DW + 1 + 1 + 1
Response fields	pslverr, prdata	last, first, pslverr, prdata

This is intentional, not a defect. The `first`/`last` bits exist so `apb4_master_stub` can carry AXI4 burst framing through the AXI4-to-APB bridge; an APB slave has no burst to frame and so has no use for them.

The two stubs are **never** connected packed-side to packed-side. They face each other across the APB bus, and the APB bus itself has no `first`/`last` signals, so no incompatibility arises. Do not attempt to wire `apb4_master_stub.cmd_data` directly into `apb4_slave_stub` — the widths differ and the field alignment differs.

10.4.4 Difference From apb4_slave

This stub is a simplified two-state FSM (`IDLE`, `XFER_DATA`) with combinational outputs. It does **not** implement the `PENABLE` rising-edge detect or the orphan-response guard present in apb4_slave. Use `apb4_slave` for anything synthesized into a real design, particularly behind a CDC.

10.4.5 Operation

1. APB master drives transaction on APB interface
2. Stub detects PSEL/PENABLE and packs command
3. Stub presents command packet on `cmd_data` with `cmd_valid=1`
4. Test responder reads command when `cmd_ready=1`
5. Test responder provides response on `rsp_data` with `rsp_valid=1`
6. Stub unpacks response and drives APB PRDATA/PSLVERR/PREADY

10.5 Timing Characteristics

This module is **sequential**: it contains clocked logic (via `always_ff` or the repository's `ALWAYS_FF_RST` macro) and therefore holds state. Outputs driven from those blocks are registered and appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

Timing closure is therefore a question of the surrounding logic's slack, not of this module's cycle count. No synthesis figures are quoted; none have been measured.

10.6 Usage Examples

```
// Instantiate APB slave stub
apb4_slave_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(16)
) u_apb4_slave_stub (
    .pclk(clk),
    .presetn(rst_n),
    // APB interface from master/interconnect
    .s_apb_PSEL(apb_psel),
    .s_apb_PENABLE(apb_penable),
    .s_apb_PADDR(apb_paddr),
    .s_apb_PWRITE(apb_pwrite),
    .s_apb_PWDATA(apb_pwdata),
    .s_apb_PSTRB(apb_pstrb),
    .s_apb_PPROT(apb_pprot),
    .s_apb_PRDATA(apb_prdata),
    .s_apb_PSLVERR(apb_pslverr),
    .s_apb_PREADY(apb_pready),
    // Packed interface to test responder
    .cmd_valid(test_cmd_valid),
    .cmd_ready(test_cmd_ready),
    .cmd_data(test_cmd_data),
    .rsp_valid(test_rsp_valid),
    .rsp_ready(test_rsp_ready),
    .rsp_data(test_rsp_data)
);

// Example: Respond to command (in testbench)
// When cmd_valid: unpack command, generate response
always_comb begin
    test_cmd_ready = 1'b1;  // Always ready
    if (test_cmd_valid) begin
        // Unpack command
        logic [DW-1:0] wdata = test_cmd_data[DW-1:0];
        logic [AW-1:0] addr = test_cmd_data[DW+AW-1:DW];
        logic pwrite = test_cmd_data[CPW-1];

        // Generate response
        test_rsp_data = {1'b0, read_memory(addr)};  // No error,
return data
        test_rsp_valid = 1'b1;
    end
end
```

10.7 Design Notes

10.7.1 Testbench Usage

This module is intended for: - System-level testbenches - Integration testing - Simple APB slave emulation - Memory model integration - Verification component integration

For production use, consider the full-featured `apb4_slave` module.

10.7.2 Response Generation

The stub expects immediate response from the test driver. For multi-cycle latency: - Test driver should buffer commands - Assert `rsp_valid` only when response is ready - Stub will hold PREADY low until response arrives

10.7.3 Packed Interface Benefits

- Simplified connection to test drivers
- Reduces signal count
- Easier integration with verification IP
- Convenient for memory models and responders

10.8 Related Modules

- `apb4_slave.sv` - Full-featured APB slave with independent interfaces
- `apb4_master_stub.sv` - Companion APB master stub
- `apb4_monitor.sv` - APB transaction monitoring

10.9 Testing

No dedicated testbench, and none is expected. This is a stub: a tie-off shell that presents the interface and drives inert values, so there is no behaviour to verify beyond elaboration. `make verilator` lints it as its own top on every run, which is the coverage that applies.

Treat any behaviour described on this page as unverified by simulation.

10.10 References

- Arm. *AMBA APB Protocol Specification*, Version 2.0, ARM IHI 0024C, Apr. 2010. developer.arm.com/documentation/ihi0024/c – ch. 3 “Transfers” and ch. 4 “Operating States”, the protocol of the wrapped APB4 block. Citation only.

- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- **Full APB Slave**: apb4_slave.md
- **APB Master Stub**: apb4_master_stub.md
- **APB5 Equivalent**: apb5_slave_stub.md
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

10.11 Navigation

- **← Back to APB Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**